

UNIVERSITY OF CALIFORNIA SAN DIEGO

Sharing is Caring: Shared Libraries for an Embedded Operating System

A thesis submitted in partial satisfaction of the
requirements for the degree Master of Science

in

Computer Science

by

Anthony Haig Tarbinian

Committee in charge:

Professor Pat Pannuto, Chair
Professor Deian Stefan
Professor Geoffrey Voelker

2025

Copyright

Anthony Haig Tarbinian, 2025

All rights reserved.

The Thesis of Anthony Haig Tarbinian is approved, and it is acceptable in quality and form for publication on microfilm and electronically.

University of California San Diego

2025

TABLE OF CONTENTS

Thesis Approval Page	iii
Table of Contents	iv
List of Figures	vi
List of Tables	viii
Dedication	ix
Abstract of the Thesis	x
Introduction	1
Chapter 1 Background and Related Work	3
1.1 Libraries	3
1.1.1 Static Linking	3
1.1.2 Shared Libraries	4
1.2 Dynamic Linking	4
1.3 Position Independent Code (PIC)	6
1.3.1 Global Offset Table (GOT)	7
1.3.2 Procedure Linkage Table (PLT)	9
1.4 Embedded Systems	10
1.4.1 Embedded Operating Systems	10
1.4.2 Memory Protection Unit (MPU)	11
1.5 Tock	11
1.5.1 Tock Kernel	12
1.5.2 libtock-c	12
1.5.3 elf2tab	13
1.5.4 Tockloader	14
1.5.5 Position-Independent-Code (PIC) on Tock	14
1.6 (There are no) Shared Libraries on Embedded Systems	17
Chapter 2 Design	18
2.1 Goals	18
2.2 High Level Design	19
Chapter 3 Implementation	22
3.1 Build Configuration	22
3.1.1 Building a shared library	23
3.1.2 Linking an application with a shared library	27
3.1.3 Incorrect PLT Generation	27
3.2 elf2tab	32

3.3	Tockloader	34
3.4	Tock Kernel	36
3.5	libtock-c	39
3.6	Walkthrough	39
3.6.1	Overview	39
3.6.2	Environment	40
3.6.3	Clone	40
3.6.4	Dependencies	41
3.6.5	Build and Flash Tock Kernel	42
3.6.6	Build a shared library	43
3.6.7	Build an application	43
3.6.8	Create shared library TAB file	44
3.6.9	Create application TAB file	45
3.6.10	Flash Shared Library to Board	45
3.6.11	Flash application to board	46
3.6.12	Verify application output	46
Chapter 4	Evaluation	48
Chapter 5	Discussion and Future Work	52
5.1	Compiler Patches	52
5.1.1	Avoid clobbering PIC base register	52
5.1.2	PLT entries use PIC base register	54
5.2	RISC-V and Rust Userspace	55
5.3	Nested Shared Libraries Calls	55
5.4	Accessing shared library global data from application	55
Chapter 6	Conclusion	57
	Bibliography	58

LIST OF FIGURES

Figure 1.	Faster Updates with Shared Libraries	2
Figure 1.1.	Static Libraries	3
Figure 1.2.	Shared Libraries	4
Figure 1.3.	Loading a Dynamically Linked Executable on Linux	5
Figure 1.4.	ELF Binary Layout	6
Figure 1.5.	DT_NEEDED Entries of ELF Dynamic Section	7
Figure 1.6.	GOT and Data Section Layout	8
Figure 1.7.	Global Variable Access with GOT	9
Figure 1.8.	PLT Control Flow	10
Figure 1.9.	Tock Architecture.....	12
Figure 1.10.	Non-relative Text and Data Sections	16
Figure 1.11.	Global Variable Access with PIC Base Register.....	16
Figure 2.1.	High Level Design Overview	21
Figure 3.3.	Example Application and Shared Library Code	22
Figure 3.6.	Saving and Restoring PIC Base Register.....	24
Figure 3.7.	Separate Application and Shared Library PIC Base Register	26
Figure 3.8.	objdump of Incorrect PLT Code Generation.....	28
Figure 3.11.	Function Pointer Declaration	31
Figure 3.14.	Bypassing PLT with Function Pointers	32
Figure 3.15.	elf2tab Relocation TOML file	33
Figure 3.16.	Shared Library TBF Header	34
Figure 3.17.	Application Code with Shared Library Function Pointer.....	35
Figure 3.18.	Application Code Disassembly	35

Figure 3.19.	GOT Before/After Tockloader Relocation	35
Figure 3.20.	Comparison of Shared Library References With and Without Function Pointers	37
Figure 3.21.	Dependency Graph of Installing Shared Libraries and Applications	38
Figure 3.22.	Sample Shared Library C Code	44
Figure 3.23.	Sample Application C Code	44
Figure 3.24.	Shared library in tockloader list.....	46
Figure 3.25.	Application in tockloader list.....	47
Figure 4.1.	Overhead of Shared Library Function Call	50
Figure 5.1.	PLT Entries Using PIC Base Register	54

LIST OF TABLES

Table 3.1.	Links to repositories	41
Table 4.1.	Shared Library Initialization Overhead	49
Table 4.2.	Shared Library Function Call Overhead	50

DEDICATION

Pat Pannuto – I’m extremely grateful for their mentorship and resourcefulness. Whenever I thought I had hit a wall, Pat would guide me to the right path. Without him, I wouldn’t have gotten the opportunity to learn so much and build something like this.

Geoffrey Voelker – for their unwavering support and advice throughout the development of this thesis.

Deian Stefan and Geoffrey Voelker – for their willingness to serve on my committee.

Tyler Potyondy – for their guidance and kindness in working with the Tock project. Tyler graciously welcomed me onto Tock and was always willing to offer a helping hand when I needed it.

Samir Rashid – for going on this crazy adventure of writing a thesis together. Their friendship and support made this process exponentially more enjoyable.

Reed O’Brien – for their life advice, mentorship and encouragement to pursue grad school.

Arpi Keshishian – for always pushing me to be my best self no matter the circumstance.

My previous teachers and professors – for shaping me into who I am today and being a constant source of inspiration. There’s too many to thank, but from UCSD I’d like to highlight Professor Ryan Kastner, Professor Michael Coblenz, Professor Steven Swanson, Dr. Tristan Knoth, Professor Joe Politz, and Professor Joseph Pasquale.

Friends, family, and loved ones – for their support and compassion.

ABSTRACT OF THE THESIS

Sharing is Caring: Shared Libraries for an Embedded Operating System

by

Anthony Haig Tarbinian

Master of Science in Computer Science

University of California San Diego, 2025

Professor Pat Pannuto, Chair

Today, shared libraries are ubiquitous in computing systems. When multiple processes share the same code, resources can be saved by removing any duplicate copies of the libraries they have in common. This technique becomes increasingly relevant in the context of multi-tenant embedded systems. Running multiple “apps” on these microcontroller-class devices strains their limited pool of persistent storage. With shared libraries, redundant library code can be eliminated, pushing the capabilities of resource constrained devices even further. This work implements the infrastructure needed to support shared libraries for a multi-tenant embedded operating system, Tock.

Updates and errata to this dissertation can be found at: atarbinian.com/ms-thesis

Introduction

Virtually all software relies on libraries to reuse existing code. It's vital for developers to build on top established software to create larger and more complicated systems. Due to the widespread nature of libraries, it's possible that a system could have hundreds or thousands of processes all using the same libraries. For this reason, many operating systems allow processes to share library code. Resources can be saved by only requiring a single copy of a library's code to exist for all processes. Additionally, if multiple processes are reading from library's data sections, those can also be shared thanks to copy-on-write pages [7].

In recent years, microcontroller-class embedded devices have become increasingly multi-tenant and networked. As a result, embedded operating systems have adapted to fit this growing need. As with traditional operating systems, embedded operating systems have multiple processes which all use library code. Just like larger classes of devices, embedded systems would benefit from sharing commonly used library code. Typically, embedded systems store and execute code sections directly on flash memory. Minimizing flash storage is always a concern on embedded systems. Developers spend plenty of time trying to minimize binary size via compiler flags or source code optimizations [52, 44]. With shared libraries, flash storage can be stretched further on the cheapest and most resource constrained devices.

If libraries are decoupled from applications, the cost of updating either application or library is reduced. Usually, both the application and library must be updated together as a part of a new firmware image. With shared libraries, libraries can be updated independently of the applications. This way all applications can enjoy updated libraries without needing to be reinstalled. Faster over-the-air updates are crucial for large scale sensor deployments connected

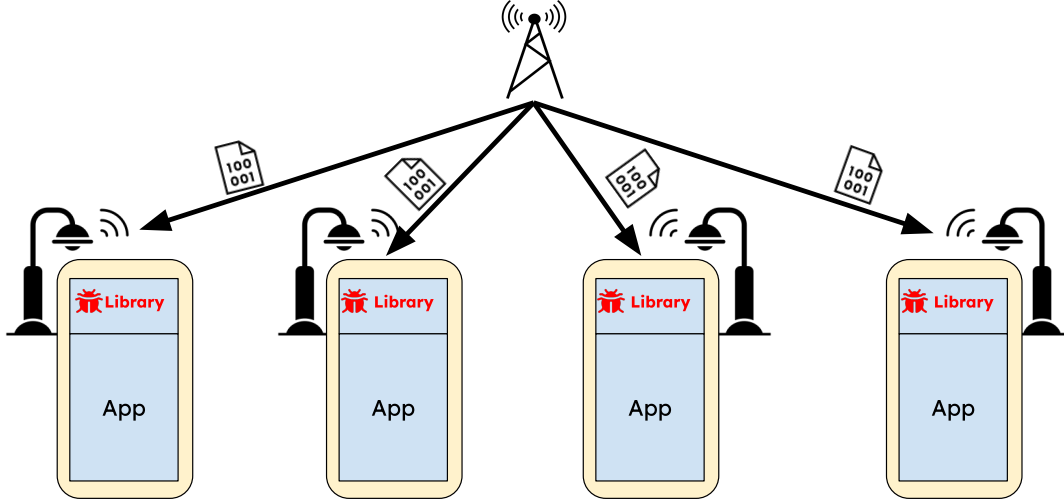


Figure 1. Faster Updates with Shared Libraries

via long range, low-bandwidth wireless networking technologies (e.g. LoRa) [54, 56]. Every bit of bandwidth is precious and breaking up libraries from applications could make a substantial difference in updating an entire network of deployed devices. Figure 1 shows a potential smart city with over-the-air security updates being sent to several smart lampposts. Shared libraries can allow for libraries to be separated from the application executable and reduce the amount of data needed to be sent wirelessly for a library update.

This work designs and implements a system to support shared libraries for the embedded operating system Tock. It examines how dynamic loading can be adapted to fit on embedded platforms. As a part of this thesis, I have updated the Tock operating system and the accompanying infrastructure to support shared libraries.

Chapter 1

Background and Related Work

1.1 Libraries

1.1.1 Static Linking

Static linking folds all library dependencies into the main executable [47]. The code of the library is copied into the final binary. This allows the linker to resolve all references to the library at link/build time. At runtime, there is nothing blocking the application from executing and interacting with the library. It already comes packaged with any needed library code in the same executable. Figure 1.1 showcases two statically linked applications. Both applications are statically linked with the same library, resulting in a duplicated copy of the library code.

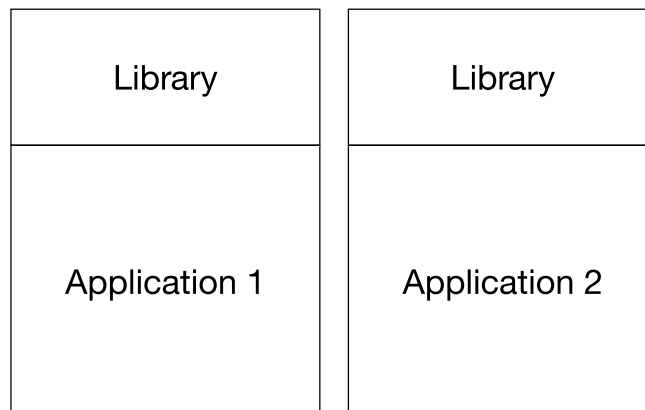


Figure 1.1. Two applications statically linked with the same duplicated library code

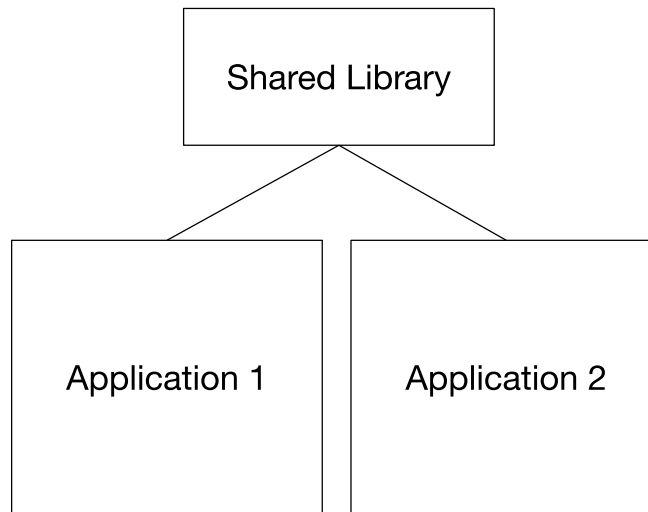


Figure 1.2. Multiple applications sharing a single copy of library code

1.1.2 Shared Libraries

Shared libraries decouple libraries from the main application binary. The library's code exists independently, outside the main binary. As a result, multiple applications can refer to the same library code as depicted in Figure 1.2. To support shared libraries, additional support is needed at runtime to resolve references to shared library symbols. Unlike static linking, all references to the library cannot be resolved at link/build time because the linker doesn't know where the shared libraries will exist at runtime. Instead, a loader is responsible for locating shared libraries and invoking a dynamic linker to patch up references to shared libraries at runtime [43].

1.2 Dynamic Linking

To understand what might be needed for shared libraries to work on embedded operating systems, it would be useful to examine the behavior other platforms exhibit when dealing with shared libraries. Take the GNU/Linux operating system for example [39]. Examining the chain of events needed to load dynamically linked executables on Linux shows the type of behavior needed at runtime to support shared libraries.

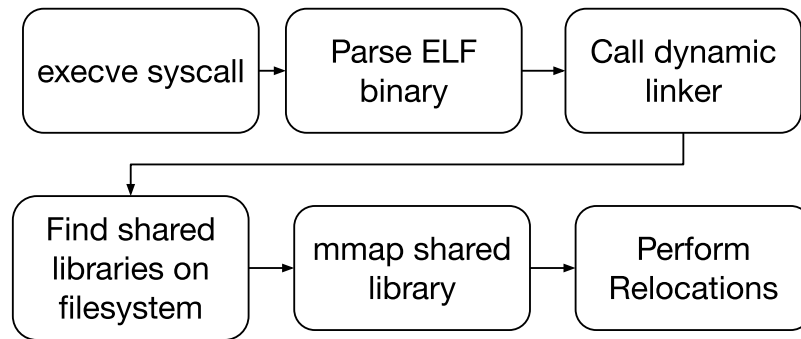


Figure 1.3. High level overview of loading a dynamically linked executable on Linux

execve System Call

For an executable to be loaded on Linux, the loading process is usually kicked off by another process trying to execute a program. One common way is via the `execve` system call [11]. One of the first things that `execve` does is parse the executable to find out what type of binary it is and subsequently how to interpret it [50]. Linux supports a variety of binary formats such as interpreted scripts and the original Unix `a.out` binary format [2].

ELF

This overview will focus on the Executable and Link Format (ELF) binary format and how Linux loads ELF executables [48]. ELF is the successor to the aforementioned `a.out` binary format and is the binary format mainly used on modern Linux systems [49]. Figure 1.4 showcases the content and layout of a typical ELF binary. For the purposes of `execve`, the most important part of the ELF file is the **Program Header Table** [30]. The Program Header Table outlines information needed by the system to prepare the program for execution. For dynamically linked programs, an important type of program header entry is `PT_INTERP` which identifies a **dynamic linker** to prepare the program for execution [49, 30]. The dynamic linker is responsible for binding any unresolved symbols and performing any needed relocations. `ld-linux.so` is the dynamic linker used for ELF binaries with `ld.so` being used for the `a.out` format [18].

To find any shared library dependencies, the dynamic linker will look in the “dynamic”

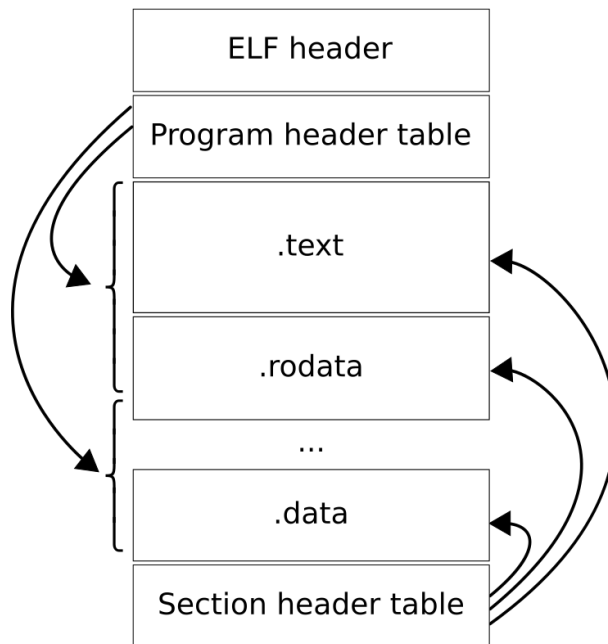


Figure 1.4. Simplified layout of a typical ELF binary – **Figure from** [14]

section of the binary. Specifically, dynamic entries of type `DT_NEEDED` will indicate the names of any required shared libraries [48, 43]. Figure 1.5 shows shared library dependencies of a Linux binary by using the `readelf` program to print out the `DT_NEEDED` entries of the binary’s dynamic section. The dynamic linker looks through these references, finds the shared library code on the filesystem. It will look through directories such as `/lib/` or `/usr/lib/` as well as additional directories using the `LD_LIBRARY_PATH` environment variable [18]. Users can also override shared libraries by using the `LD_PRELOAD` environment variable to point to shared objects that implement the same functions. Now that the dynamic linker has found the shared libraries on the filesystem it can map them into the memory of the new process. However, this begs the question, where in the process’s address space does the library get mapped?

1.3 Position Independent Code (PIC)

The dynamic linker can take advantage of virtual memory to map shared libraries into a process’s address space. However, the dynamic linker cannot know the exact location ahead of


```
Dynamic section at offset 0x39c8c0 contains 31 entries:
  Tag                Type                Name/Value
  0x0000000000000001 (NEEDED)           Shared library: [libgcc_s.so.1]
  0x0000000000000001 (NEEDED)           Shared library: [libm.so.6]
  0x0000000000000001 (NEEDED)           Shared library: [libc.so.6]
  ...
```

Figure 1.5. Output of `readelf --dynamic <binary>` which indicates shared library dependencies via `DT_NEEDED` entries of the ELF dynamic section.

time ([19]). The placement of shared libraries into a process’s virtual address space depends on the number of libraries being used by the process as well as the order in which those libraries were loaded. Either of these can change where a library ends up being mapped in the address space. Additionally, security defenses such as ASLR (Address Space Layout Randomization) further randomize where libraries get mapped in memory [1] making it impossible to know where a shared library will be mapped at build time.

To deal with this uncertainty, one solution is to make the shared library code be **position independent**. In other words, it should function no matter where in memory it is loaded. Many parts of the code are already position independent. Accesses to local variables and function arguments will still work since both are relative to the stack pointer register (sp). Everywhere else, references to absolute addresses need to be removed. Local jumps and branches can be modified to be relative to the program counter register (pc). However, some extra infrastructure is needed to support global data accesses and function calls

1.3.1 Global Offset Table (GOT)

Accessing global data (e.g. global variables) in a position independent manner becomes more complicated. The location of where the data section will be loaded into memory can’t be known at compile time. As such, the dynamic linker must resolve global data references when the library is loaded. The dynamic linker could go ahead and individually resolve every global data reference in the code [45]. However, this is costly at load time. One solution to reduce the

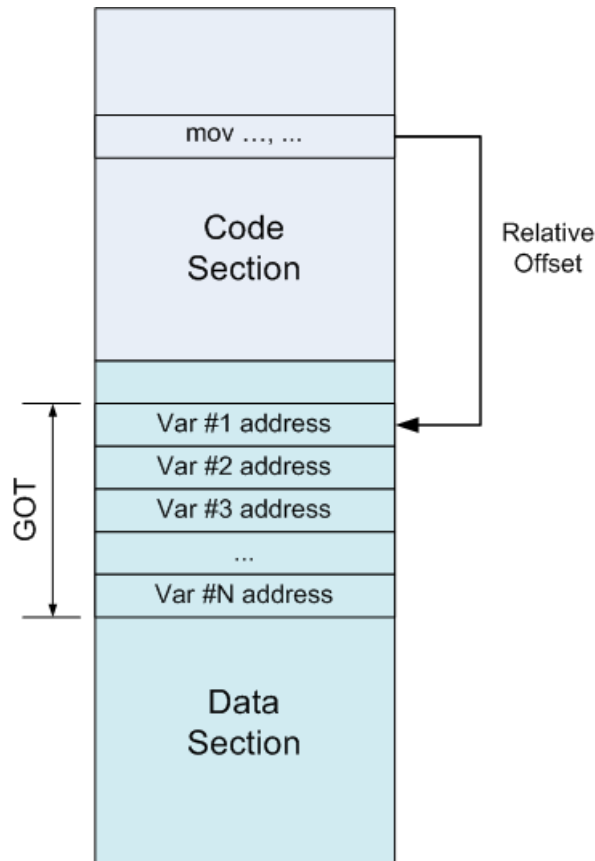


Figure 1.6. GOT and Data Section Layout – **Figure from** [46]

number of relocations is via a level of indirection. A **Global Offset Table** (GOT) achieves this level of indirection for global data accesses. The GOT will act as a lookup table which will store the actual location of global data members. See Figure 1.6 for an overview of how the GOT entries are laid out. The compiler will replace each global variable reference to a lookup in the GOT. After relocation, the GOT will contain the actual address to global variables.

On platforms such as x86 Linux, the compiler takes advantage of the fact that text sections are relative to data sections [46]. As such, the compiler can generate code that performs PC-relative (program counter relative) accesses to the GOT. The program counter will point to the next instruction to be executed, which lies in the text section. Since data is relative to text, the compiler can make accesses to the GOT relative to the current PC. Figure 1.7 showcases the difference in compiler generated code for accessing a global variable with and without the GOT.

```

int var;

void foo(void) {
    var = 1;
}

foo:
; load address to var
ldr r3, [pc, #4]
movs    r2, #1
; write 1 to var
str r2, [r3, #0]
bx  lr

foo:
; load PC-relative offset to GOT
ldr r3, [pc, #12]
; calculate actual GOT address
add r3, pc
; load var's offset inside GOT
ldr r2, [pc, #12]
; load location of var from GOT
ldr r3, [r3, r2]
; write 1 to var
movs    r2, #1
str r2, [r3, #0]
bx  lr

```

Figure 1.7. Global Variable Access Without and With GOT. The top shows sample C code. The bottom left and right are the ARM assembly of two variations of the compiled C code. The left side indicates access to a global variable without using the GOT. The address to the variable *var* is stored in the text section and is relocated at runtime. On the right side, the access to the same variable is done via the GOT. First the PC-relative offset to the GOT is loaded and used to calculate the actual address of the GOT (using the current pc). Then the offset to *var*'s entry within the GOT is loaded. Using the address of the GOT and the offset of *var* within the table, the code can access the address where *var* exists in the data section. Once this address is found, it can access the variable itself. This example was slightly modified and lifted from [59].

1.3.2 Procedure Linkage Table (PLT)

Function calls to shared library also need to be resolved at runtime. This is commonly achieved via a jump table or sometimes called **Procedure Linkage Table** (PLT) which resides in the executable text section of a program. Instead of calling the function directly, the application jumps into an entry of the PLT. The PLT contains executable code which takes care of jumping to the actual function [46]. The PLT will refer to a second GOT which exclusively contains addresses for functions referred to by PLT entries (frequently called the `.plt.got` section) [43].

One optimization made possible thanks to the PLT is lazy binding. With lazy binding, relocations for functions doesn't have to be done at program load time. Instead, they can occur

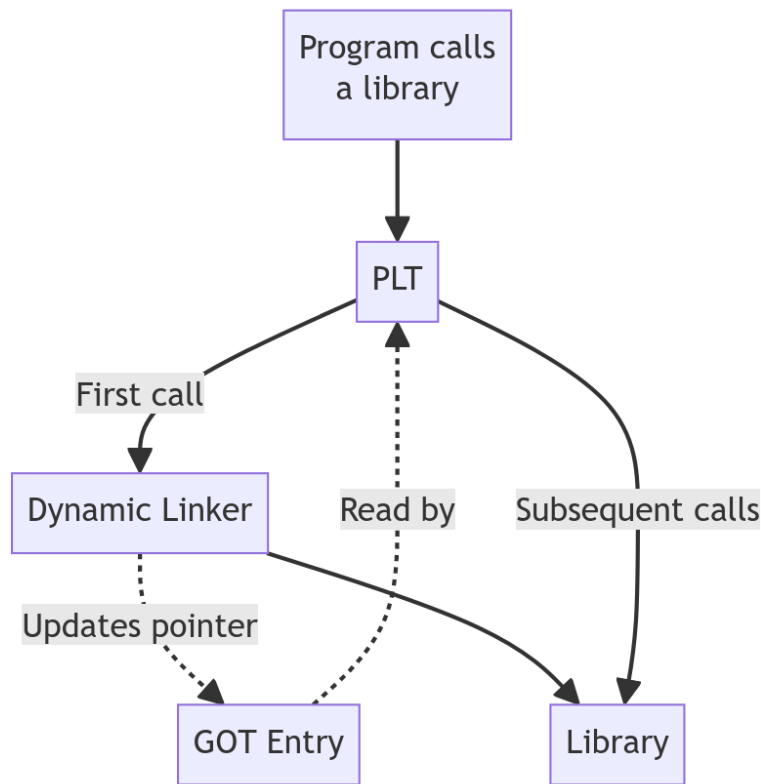


Figure 1.8. PLT Control Flow With Lazy Binding – **Figure from** [43]

the first time a function is called. This optimization can save plenty of time since functions which never get called will never need relocation. On the first call, the PLT will invoke a resolver routine (usually a part of the dynamic linker) which will find the location of the target function symbol [46]. Once the location of the function is found, the address is cached in the GOT. Then, on subsequent calls to the function, the PLT will simply use the cached value in the GOT to jump directly to the function in question. Figure 1.8 walks through the control flow of calling a function with lazy binding the first time compared to subsequent calls.

1.4 Embedded Systems

1.4.1 Embedded Operating Systems

Traditionally, embedded systems were simple enough to only require a single program binary. They were usually only responsible for doing one simple task, so running multiple

applications was never a priority. However, in recent years, embedded software is becoming increasingly networked and complex. As this trend continues, the need for multiprogramming increases. Some well-known embedded operating systems include: FreeRTOS, Zephyr, or TinyOS.

1.4.2 Memory Protection Unit (MPU)

Embedded devices lack the fully-fledged memory management unit (MMU) that desktop and server class machines enjoy. As a result, embedded devices cannot support virtual memory and are stuck with the physical address space. In lieu of an MMU, many hardware manufacturers have chosen to include a Memory Protection Unit (MPU) instead [41]. An MPU is a stripped down version of an MMU which only provides memory protection. It is a programmable piece of hardware that allows the firmware to define *regions* and assign memory access permissions to them [41].

1.5 Tock

Tock is an embedded operating system designed to run mutually distrustful applications on microcontroller-class embedded devices [53]. It is designed with kernel process isolation and fault tolerance in mind. Tock has been adopted primarily in secure-root-of-trust applications due to its strong isolation guarantees.

Tock, unlike other embedded operating systems, truly focuses on multiprogramming between multiple, isolated computation units. The Tock kernel is an entity which exists separately from each userspace application. Other embedded operating systems, such as Zephyr and FreeRTOS, are more akin to a “library OS” which is simply compiled together with the applications to form one large binary [60] [40].

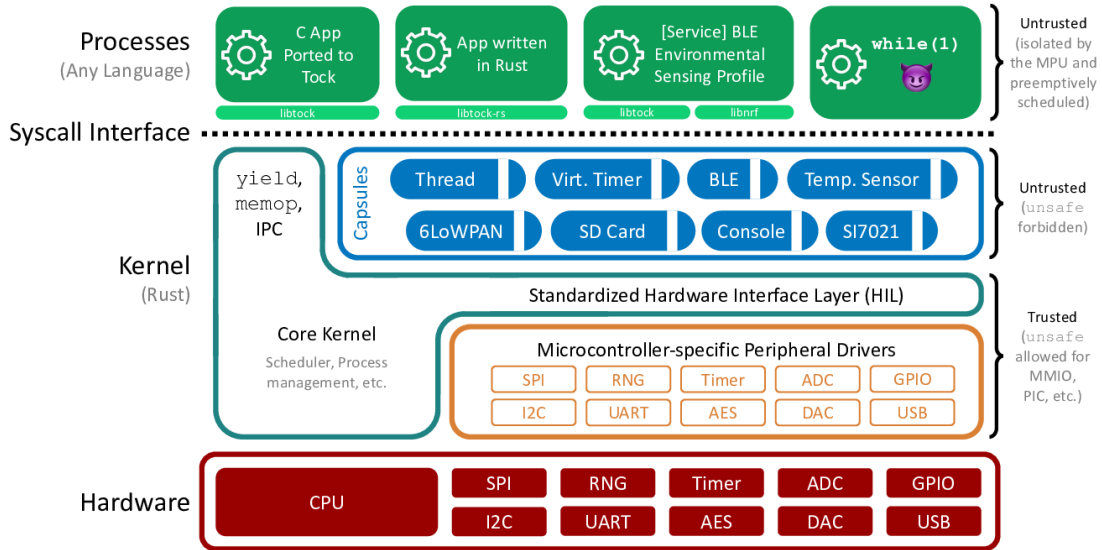


Figure 1.9. Tock Architecture – **Figure From** [17]

1.5.1 Tock Kernel

The Tock kernel is responsible for scheduling userspace processes, handling system calls and virtualizing underlying hardware resources. The kernel is written in Rust to take advantage of Rust’s memory and type safety.

The kernel is split into a few components. The core kernel is a small, trusted codebase which is responsible for scheduling and process management. The core kernel is permitted to use `unsafe` blocks of Rust code to perform critical tasks such as interacting hardware registers [17]. Capsules are untrusted parts of the kernel which implement logic to carry out system calls. They are still written in Rust, but are not permitted to use `unsafe` blocks. See Figure 1.9 for an overview of Tock’s architecture.

1.5.2 libtock-c

`libtock-c` is one of Tock’s userspace libraries. `libtock-c` targets applications written in C/C++ and provides a convenient interface to make system calls to the Tock kernel [20]. The `libtock-c` repository also allows developers to easily build and link libraries into their applications. Some of these libraries include `newlib` for a C standard library implementation,

lr1110 for LoRaWAN and Wi-Fi scanning, and libopenthread for interoperability with Thread networks [20].

crt0

libtock-c includes a startup routine for all applications called crt0. crt0 includes any code that runs before the main function is called [55]. It performs any necessary setup and initialization needed for an application to run. Section 1.5.5 will discuss the role crt0 plays in libtock-c applications.

Linker Script

libtock-c also provides a linker script which is used while building applications. A linker script specifies the format and layout of an executable binary [22]. libtock-c creates a linker script which follows the syntax accepted by the GNU linker (ld) [42]. One interesting purpose of the libtock-c linker script is to place certain sections in flash memory while others go in SRAM. It indicates which sections belong to flash by offsetting them by the address 0x80000000. This way, any tools reading the binary can tell if an address was meant for flash or memory and perform the appropriate relocations once the location of application in flash is known.

1.5.3 elf2tab

elf2tab is a tool developed for the Tock ecosystem to aid in converting ELF executable files into Tock Application Bundles (TABs). A TAB is a fat binary, in other words, a collection of executables that target all architectures that Tock supports [6, 13]. This means that TABs can be board-agnostic as they include every version of the binary that any board would need. A TAB is quite literally a tar archive of the executable file of several architectures [36].

elf2tab is also responsible for prepending each binary with the Tock Binary Format (TBF) header [37]. The TBF header contains metadata needed by kernel and tools like Tockloader. The TBF header contains a set of TBF objects. Each TBF object has a base header followed

by optional headers. The optional headers are structured as TLVs (type-length-values) [38]. Tock supports TLVs that indicate the entrypoint of an application's program, package name, permissions and much more.

1.5.4 Tockloader

Tockloader is a tool used to aid in flashing Tock applications onto actual boards. Tockloader takes the TAB files created by `elf2tab`, finds the architecture needed by the current board, and flashes the binary to the board using a tool like JLink or OpenOCD [16, 29]. Tockloader can also change the layout of existing applications on the board by reshuffling their positions. This way it can lay out applications as the user desires or to avoid fragmentation.

1.5.5 Position-Independent-Code (PIC) on Tock

As Tock has no virtual memory, application code is located in the physical address space and cannot be known until the binary is flashed. This means that application code must be built in a position independent fashion as discussed in Section 1.3. This translates to requiring all `libtock-c` applications to be compiled with the GCC flag `-fPIC` [6, 5]. As a result, GCC avoids absolute addressing and generates code which accesses global data via the GOT. Since `libtock-c` applications are currently statically linked with libraries, there is no need for a PLT. However, with the introduction of shared libraries the PLT will become of relevance again (Section 3.1.3).

However, PIC isn't exactly the same for embedded devices. On platforms such as x86 Linux, one could assume that text sections are relative to data sections. This allows the GOT to be accessed in a PC-relative fashion as discussed in Section 1.3.1. On embedded devices, this isn't the case. Typically, embedded devices will execute code from directly flash memory and use SRAM to store data sections. See Figure 1.10 for the typical layout of the address space with non-relative text and data sections. This means text and data sections are in different parts of the physical address space with no fixed offset between them. The location of the

data sections is only known when memory is allocated at runtime. To accommodate this, Tock takes advantage of a few GCC flags which avoid PC-relative GOT addressing. These flags include: `-mno-pic-data-is-text-relative`, `-mpic-register=r9`, `-msingle-pic-base` [6, 3]. The first flag (`-mno-pic-data-is-text-relative`) tells GCC to not assume a fixed distance between text and data sections. The next flag (`-mpic-register=r9`) changes every access to the GOT to be done with the help of a special register (in this case r9). This register will henceforth be referred to as the PIC base register. The PIC base register stores the base address of the GOT. With this, code can access entries to the GOT by reading memory relative to where the PIC base is pointing to. See Figure 1.11 for a comparison of GOT accesses with and without the PIC base register. The last flag (`-msingle-pic-base`), tells GCC to treat r9 as a read-only register and to assume that it will be initialized correctly to point to the GOT.

In `libtock-c`, `crt0` is responsible for setting up the PIC base register among other things. After setting up the stack, `crt0` sets the value of the PIC base register (r9) to the actual location of the GOT in memory [20]. In addition, `crt0` copies the GOT and data sections from the binary to application memory. It also sets up the stack and zeroes out BSS memory. To know the location of GOT, data and BSS in the binary `crt0` uses the `crt0_header`. The `crt0_header` is the first part of any `libtock-c` ELF binary which contains addresses that indicate the location of important symbols within the binary (e.g. `.got`, `.data`). This `crt0_header` is assembled by the linker when it processes the custom `libtock-c` linker script. The linker script defines the `crt0_header` to be at the start of the binary which makes it trivial for `crt0` to access the header at the start of where the binary has been stored in flash memory. To know the location of where the application's memory has been allocated, the kernel passes this information via registers when it calls `crt0`.

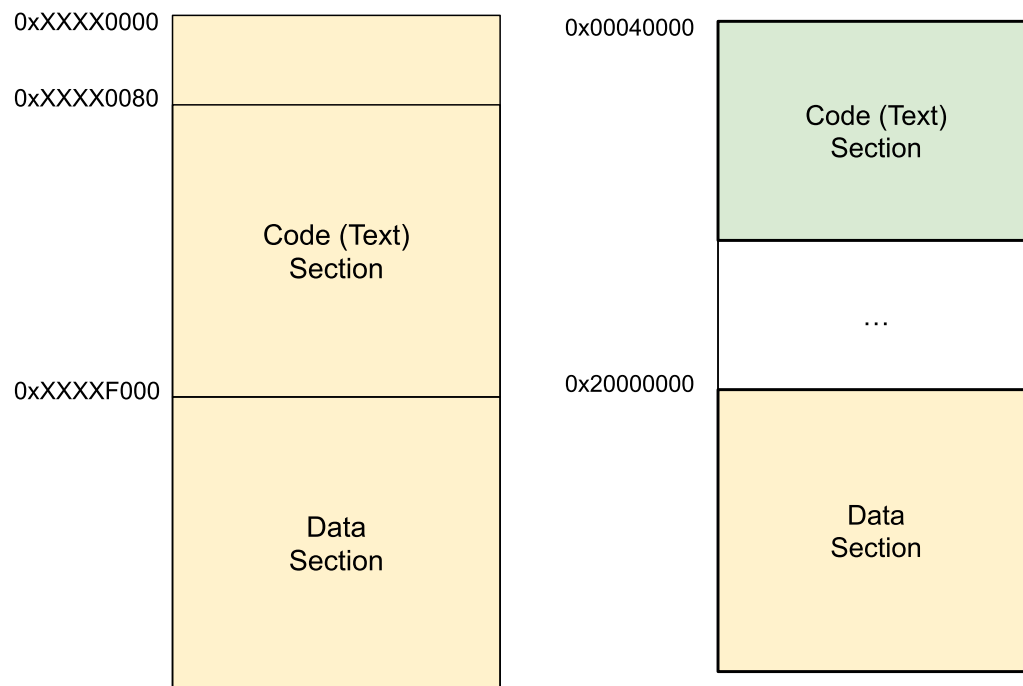


Figure 1.10. The left showcases the typical layout of text and data sections within a process running on a desktop/server operating system with virtual memory. Note the fixed offset between the text and data sections as the OS can place them next to each other in the virtual address space. The right depicts the typical layout of text and data on embedded platforms. The green indicates flash memory while the yellow is SRAM. Text is placed in flash which is in a completely separate part of the physical address space. Data sections are placed in SRAM. Both are affected by other applications running on the board. In addition, the lack of a virtual address space makes the distance between them impossible to know at compile-time.

```

foo:
; load PC-relative offset to GOT
ldr r3, [pc, #12]
; calculate actual GOT address
add r3, pc
; load var's offset inside GOT
ldr r2, [pc, #12]
; load location of var from GOT
ldr r3, [r3, r2]
; write 1 to var
movs r2, #1
str r2, [r3, #0]
bx lr

foo:
; load var's offset inside GOT
ldr r2, [pc, #12]
; use the PIC base register (r9) to
; access var's GOT entry
ldr r3, [r9, r2]
movs r2, #1
; write 1 to var
str r2, [r3, #0]
bx lr

```

Figure 1.11. Global variable access in a PC-relative manner (left) and with PIC Base Register r9 (right). Example continued from Figure 1.7

1.6 (There are no) Shared Libraries on Embedded Systems

As mentioned in Section 1.5, Tock is unique among embedded operating systems for truly multiprogramming separate executables on the same hardware. Most embedded operating systems statically link their kernel with the code of all their user applications. Even besides embedded operating systems, embedded devices are usually only responsible for performing a single task via a single unit of execution where there is no other unit to share execution with. For these reasons, other embedded operating systems typically don't support shared libraries. To my knowledge, no other embedded operating systems support shared libraries among multiple applications. There are a few posts on community forums for the Zephyr and FreeRTOS projects which discuss how they statically link libraries in favor of sharing them [34, 25].

Chapter 2

Design

This work aims to provide support for shared libraries for multiple applications running in the context of an embedded operating system. This means that every application will be able to invoke library functions, while requiring only a single copy of each library's code in non-volatile storage.

2.1 Goals

1. **Efficiency** Crossing between an application and a shared library function should incur as little overhead as possible. Library functions may be invoked many times during the life cycle of a program, so any additional operations should be minimized. Additionally, any overhead required at runtime to initialize a shared library to be used should be minimized as well. However, since this will only happen once on application startup it isn't as high of a priority.
2. **Security** When an application is being executed, it should only have permissions to read and execute code from shared libraries that it is designated to use. It should not be permitted to execute code from a shared library that it was never linked with. Library code may be sensitive and should not be openly available to unauthorized applications. Shared libraries can also widen the attack surface for code reuse attacks such as return-oriented-programming (ROP) and return-to-libc [57]. These attacks can achieve arbitrary

code execution by chaining together small pieces of code, or gadgets, from the application binary, libraries, etc. By restricting shared library access to only what is strictly needed, the amount of code available for these attacks can be minimized.

3. **Compatibility** Using shared libraries should ideally require no changes to application code. The developer should just have to build their code and libraries using the appropriate build configuration and let the infrastructure handle the rest.
4. **Isolation** As applications exist within an operating system, they may be running alongside other applications creating a need for isolation between each execution unit. In the case where multiple applications are sharing the same library, application state must be kept separate. It's important to note that libraries can keep their own state (e.g. local/global variables). With shared libraries, each application using a given library must maintain their own copy of library state. The operating system will already maintain separate copies of library local variables just like the application's own. Since library local variables are stored on the same stack, when the operating system changes the stack pointer on context switch, each process retains its own copy. However, state in the data section (e.g. global variables) will require extra mechanisms on embedded platforms.

2.2 High Level Design

Just like on desktop/server operating systems, embedded operating systems will need a mechanism to perform relocations to perform function calls to shared libraries. Unlike desktop/server operating systems, embedded operating systems lack virtual memory and the ability to load code at arbitrary locations in a process's address space. As such, the design must take into account the fixed physical address space of embedded devices. Figure 2.1 showcases a high level overview of the design and what's involved in going from the build stage to runtime.

Since the location of shared libraries in flash memory cannot be known at build time, shared libraries must be built as position independent code (PIC) as discussed in Section 1.3.

This means that shared libraries will need to access their global data members via a Global Offset Table (GOT).

To perform the relocations needed to access shared library symbols, one option is to implement a dynamic linker similar to the ones used on desktop/server operating systems (Section 1.2). Before the application starts executing, the dynamic linker would find the shared libraries the application depends on and perform the necessary relocations. Unlike desktop/server operating systems, embedded operating systems typically lack a file system, so the dynamic linker would need to traverse flash memory to find the shared libraries.

However, on embedded devices there is an opportunity for optimization. Once the shared library is built and flashed onto the device, the location of the shared libraries code will be at a constant physical address. Recall that embedded devices typically execute code directly from flash memory which removes the need for code sections to be loaded into memory (Section 1.5.5). Since the code of the shared library will always be at the same location in flash memory, function calls to shared library functions can be resolved at "flash time". In other words, references to the shared library functions can be resolved to directly point to the actual function addresses in flash memory for all processes. This optimization eliminates the overhead of resolving shared library symbols at runtime and further bridges the performance gap between statically linked and dynamically linked libraries.

Now that both the application and shared library code has been flashed and references to shared library symbols have been resolved, the next step is to allocate memory. The kernel will allocate memory for any shared libraries that the application needs. Each application will get its own copy of the shared library's data section to maintain isolation between applications.

Then the C runtime takes care of copying the GOT and data sections of the shared library into application memory. It also sets up the PIC base register to point to the GOT of the shared library in application memory which allows shared library functions to access their own global data members.

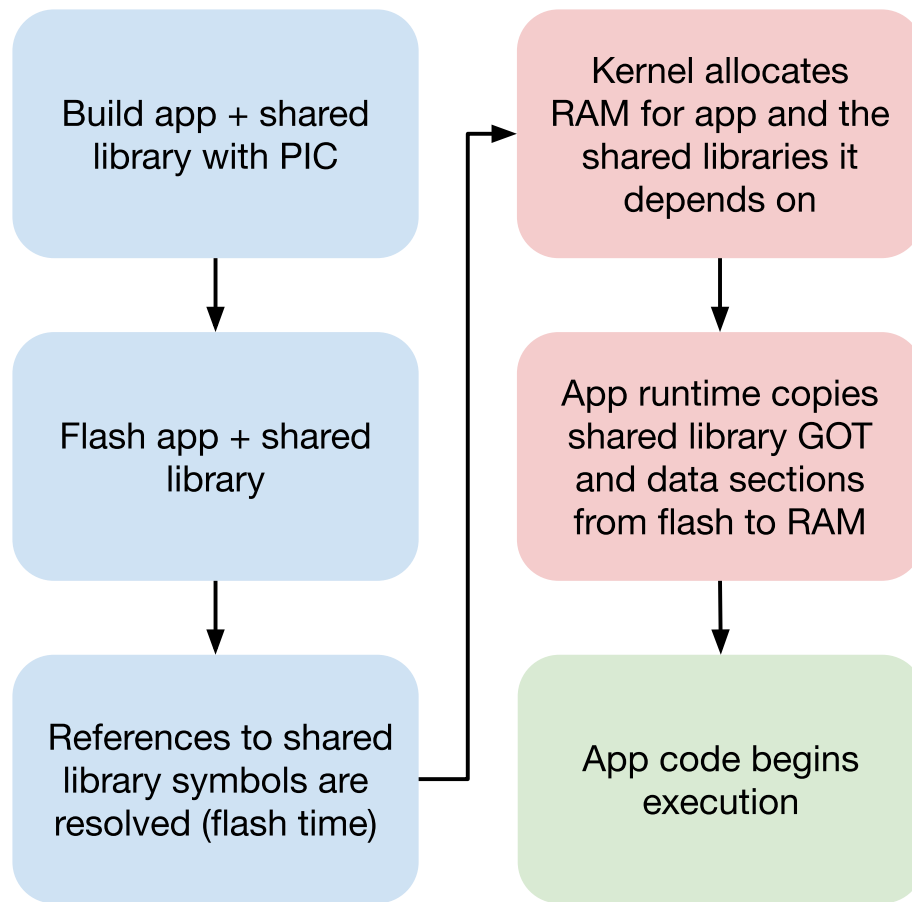


Figure 2.1. High Level Design Overview:

The blue boxes indicate steps that happen at build/flash time while the orange boxes indicate steps that happen at runtime when the application is started. Finally, the green box is the actual application code itself once any startup procedures have completed.

1. The application and shared library code are built separately as position independent code (PIC).
2. Both the application and shared library binaries are flashed onto the device.
3. When the application is flashed, references shared library code are resolved. The location of the shared library code is known in flash, so any references in the application binary can be resolved.
4. When the application is started, the operating system allocates memory for the application and any shared libraries it depends on.
5. Before the app starts up, the app's runtime copies the GOT and data sections from the shared library binary in flash into application memory.
6. The application can start executing and making shared library function calls.

Chapter 3

Implementation

I have updated the Tock operating system and its supporting tools to support shared libraries. Applications are able to call into shared library code where the library code can perform its own global data accesses. With that being said, the implementation is limited in scope and exhibits a few limitations. This implementation targets applications and shared libraries written in the C programming language, compiled with GCC for the 32-bit ARM architecture. This chapter will examine the modifications that were needed to support shared libraries and the challenges that came with them.

3.1 Build Configuration

As a running example, take the following shared library and application showcased in Figure 3.3. It demonstrates an application calling a shared library function which in turn accesses its own global variable.

```
#include <libtest.h>                                     int glob = 42;

int main(void) {                                          int lib_func(int x) {
    int result = lib_func(23);                            return x + glob;
    return result;                                         }
}
```

Figure 3.1. Application (*main.c*)

Figure 3.2. Shared Library (*libtest.c*)

Figure 3.3. Example Application (left) and Shared Library (right) C code

For shared libraries to be shared by multiple applications they must be decoupled from the application binary. This means at the build stage shared libraries must be built separately from applications.

3.1.1 Building a shared library

First and foremost, the `-shared` flag is passed to GCC which will “produce a shared object which can then be linked with other objects to form an executable” [21].

As with Tock applications, shared libraries also must be position independent. There’s no way to tell at build time how many shared libraries will exist or where in flash memory they will be placed by Tockloader. As such, shared libraries should be compiled with the `-fPIC` GCC flag to generate code that only uses relative addressing [5].

Shared libraries also need to access their own data members (e.g. global variables). As mentioned in Section 1.5.5, to support global data accesses in position independent code, additional compiler flags and mechanisms are needed [6]. These were:

- `-mno-pic-data-is-text-relative`
- `-mpic-register=r9`
- `-msingle-pic-base`

Recall that `-mno-pic-data-is-text-relative` flag explicitly tells the compiler to not assume that data and text sections will be relative to each other. As such, this flag prevents global data accesses from being done using program counter (pc) relative addressing. Instead, the compiler generates code that accesses data members by going through the Global Offset Table (GOT).

PIC Base Register

When examining how Tock applications access the GOT in Section 1.5.5, the notion of a special PIC base register was established with the flags `-mpic-register=r9` and

-msingle-pic-base. The flag -mpic-register=r9 designates r9 as the PIC base register which means that it will store the base address of the GOT and -msingle-pic-base marks r9 as read-only for the code that GCC generates. With these two flags, Tock can set r9 once during crt0 and the application subsequently access the global data it needs via GOT lookups.

These flags work fine in isolation, but the addition of shared libraries throws a wrench into the works. If shared libraries were also compiled with the same flags, the special PIC base register (r9) would be conflicting between the address of the application's GOT and the shared library's GOT. Both the application and shared library need access to their separate, respective GOT to access their own data section, so simply sharing the same PIC base register will not suffice.

```
main:
...
push {r9} ; save app PIC base
bl lib_func
pop {r9} ; restore app PIC base
...
```

Figure 3.4. Application

```
lib_func:
ldr r9, ??? ; restore lib PIC base
... ; do work
str ???, r9 ; save lib PIC base
bx lr ; return
```

Figure 3.5. Shared Library

Figure 3.6. Pseudo ARM assembly of the PIC base register r9 being saved and restored at the function call boundary between applications and shared libraries

One potential solution is to switch the value of the PIC base register when execution crosses the boundary between applications and shared libraries. The application calling the shared library function would be responsible for saving the value of its PIC base register (e.g. by pushing it onto the stack). Then when control flow transfers to the shared library, it will restore the value of its own PIC base. When the library call completes, the library saves its PIC Base and the application restores its own. See Figure 3.6 for an example of this interaction. An alternative approach could implement this functionality using a jump table, similar to the PLT. Every time the application calls a shared library function, it first jumps to a bit of code responsible for saving the PIC base before moving onto the actual library code. With regard to efficiency, this approach

adds additional overhead on every shared library function call to save and restore the respective PIC base registers.

While this swapping of the PIC base seems promising, it raises a few questions. Where will the shared library restore its PIC base register value from? It can't be stored in the data section since the PIC base register is needed to access the data section. It can't be in the text section since the location of the GOT is known at runtime when memory is allocated. Additionally, GCC did not seem to have compiler flags which would generate the extra code needed to save and restore the PIC base on every shared library call [5].

As a means to achieve a working proof of concept, the idea to juggle between multiple PIC base register values was put on hold. Instead, this implementation opts to set aside other registers to act as the PIC base for shared libraries. This way, both the application and shared library can coexist without clobbering the other's PIC base register. For example, if r9 is reserved as the application PIC base, r10 would be the first shared library's PIC base, r11 for the second and so on. See Figure 3.7 for an overview of data accesses with separate application/library PIC base registers. As for initializing the shared library's PIC base register, the application startup (crt0) routine can set the PIC base registers for each library that it is using. The shared library would be compiled with the `-msingle-pic-base` option which would make GCC avoid using its PIC base register (e.g. r10) in any code generation of the library. However, this wouldn't stop the application from having code that clobbers the library's PIC base register since the application is compiled separately. One way to have GCC avoid using the library's PIC base register is to use a GCC feature called *Global Register Variables* [12]. It will reserve a register to be used exclusively for a specific global variable. As long as the application doesn't use this global variable, the register will remain untouched by GCC.

The main drawback to this approach is that the number of shared libraries a single app can use is limited to how many registers are available to be reserved to be PIC bases. There's thirteen general purpose registers on an architecture like ARMv7-M [4]. Several are already reserved by the calling convention to be used for passing function arguments. r9 is already taken

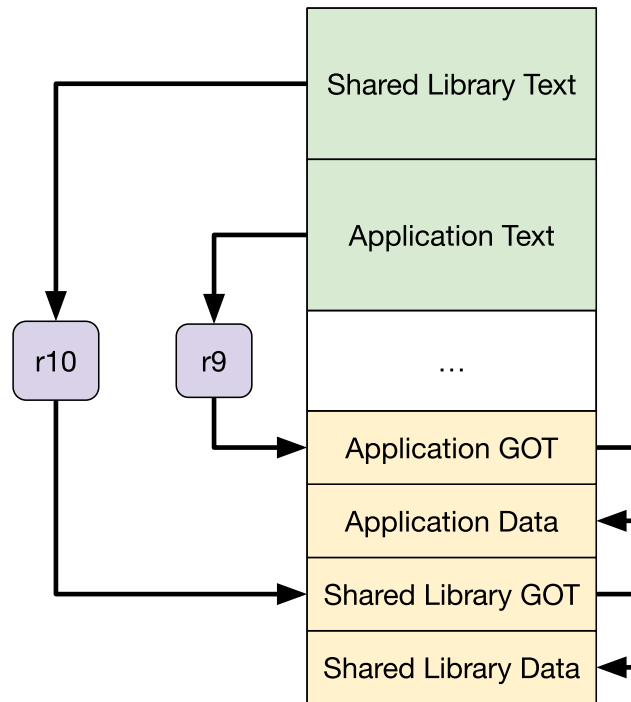


Figure 3.7. Overview of an application and a shared library both accessing their own data sections via separate PIC base registers. Green represents flash memory and yellow represents SRAM.

as the PIC base for the application which doesn't leave too many registers left for shared library PIC bases. Additionally, with fewer registers available for the compiler to use, performance may take a hit as the compiler might generate code which performs more memory accesses due to the limited pool of registers available.

In summary, these are the additional flags chosen for compiling shared libraries (Note that the register chosen for `-mpic-register` isn't required to be `r10`. It must be different from `r9` and the PIC base of any other shared library linked with a given application):

- `-fPIC`
- `-shared`
- `-mpic-register=r10`

- `-msingle-pic-base`
- `-mno-pic-data-is-text-relative`

3.1.2 Linking an application with a shared library

Building and linking a Tock application with a shared library is mostly the same as without shared libraries. First, the standard Tock application PIC flags are needed: `-fPIC -mpic-register=r9 -msingle-pic-base -mno-pic-data-is-text-relative` [6].

Next, the shared library binary must be linked into the main application’s executable. Note that even though shared libraries are loaded dynamically at runtime, they still need to be referenced at link time. This is to allow the linker to verify that the libraries contain all the symbols referenced by the application [58]. Linking the shared library to the executable follows the same conventions expected by GCC on any other architecture or platform. A flag specifying the name of the library to be linked must be included, and it follows the format of `-l<library>` where `<library>` is the name of the library file without the prefix “lib” and file extension. If the shared library was a file named `libtest.so`, the flag `-ltest` would be passed into GCC. In addition, the `-L` or `--library-path` flag is needed, so the linker can find the library on the filesystem.

3.1.3 Incorrect PLT Generation

With the aforementioned build configuration, the application executable will successfully compile and link with a shared library. However, after inspection of the compiled executable, there seem to be issues with the code generation of PLT entries.

Recall the workings of the PLT from Section 1.3.2, when a shared library function is called, execution first transfers to code from an entry in the PLT. Once the location of the function is resolved, the PLT code will look up the location of the function in the GOT and jump to that address.

The code in the PLT needs to access entries in the GOT to know where the function exists. This raises the question: how does PLT code access the GOT? PLT code is just like the other text sections and is executed directly from flash memory. As with text, when executing PLT code from flash, any lookups to the GOT cannot be done in a PC-relative (relative to the program counter register) fashion since the offset between text and data sections can't be known at compile time. This issue was solved previously by using the following GCC flags: `-fPIC -mpic-register=r9 -msingle-pic-base -mno-pic-data-is-text-relative` as described in Section 1.5.5 [6]. The important outcome of these flags is that GCC will generate code that doesn't assume the distance between text and data is fixed and that it will access the GOT using the PIC base register (r9) instead of PC-relative addressing. However, the code generated by `arm-none-eabi-gcc` version 14.2.1 only generates PC-relative GOT accesses in the PLT and not the expected r9 relative accesses. Interestingly, other accesses to the GOT (e.g. access global variables) all compile correctly and use the PIC base register to do so. It's only the GOT accesses within the PLT entries which are PC-relative.

```

80000072 <main>:
80000072: 2017      movs r0, #23
80000074: f001 baa4  b.w 800015c0 <_etext+0x10>
...
800015b0 <.plt>:
800015b0: b500      push {lr}
800015b2: f8df e008  ldr.w lr, [pc, #8] @ 800015bc <.plt+0xc>
800015b6: 44fe      add lr, pc
800015b8: f85e ff08  ldr.w pc, [lr, #8]!
800015bc: 7ffff2d4  svcvc 0x00fff2d4
; ***function call starts by jumping to here***
800015c0: f24f 2cd0  movw ip, #62160 @ 0xf2d0
800015c4: f6c7 7cff  movt ip, #32767 @ 0x7fff
800015c8: 44fc      add ip, pc
800015ca: f8dc f000  ldr.w pc, [ip]
800015ce: e7fc      b.n 800015ca <.plt+0x1a>

```

Figure 3.8. objdump of main function and .plt section which displays the assembly generated from compiling the C code in Figure 3.1 [28]

Figure 3.8 showcases the ARM assembly generated by GCC when compiling the application C code from Figure 3.1. The following walks through this code starting from the label `main`.

1. The code in `main` is relatively simple as it loads the first argument (the constant 23) into `r0` and performs a jump to address `0x800015c0`.
2. Looking at address `0x800015c0` leads to code in the `.plt` section. The lines `movw ip, #62160` and `movt ip, #32767` set register `ip` to the value `0x7ffff2d0` (`movt` sets the high 16 bits of the destination while leaving the lower 16 untouched) [24].
3. The next line at address `0x800015c8` increments `ip` (which is `0x7ffff2d0`) by the program counter (which is `0x800015ca`). This line sets `ip` to the value of an entry in the GOT by using an offset relative to the program counter. This is where the problem with this code arises. As mentioned above, PC-relative accesses to the GOT cannot be done since the distance between text and data sections is not known at compile time (which is when this code was generated).
4. Finally, line `0x800015ca` dereferences the new value of `ip` and sets the program counter. Recall in the previous line where `ip` is meant to hold the address of a GOT entry. By setting program counter, this line attempts to jump to the address held in the GOT entry that `ip` is pointing to. However, as noted in the previous line, the offset between this code (which is in flash) and the GOT section (which will exist in memory) cannot be known at this time.

There exists a post to the GNU ARM Embedded Toolchain Q&A which describes the same phenomenon [32]. A user reported using the same flags (`-msingle-pic-base -mpic-register=r9 -mno-pic-data-is-text-relative`) and saw that their PLT code also exhibits PC-relative addressing to the GOT, instead of using the PIC base register `r9`. The post

includes ARM assembly similar to that of the PLT entry in Figure 3.8. Unfortunately, this post was made in 2018 and has no replies.

One can assume this is a unintended behavior on the part of GCC. Why should only PLT got accesses be PC-relative while all other ones respect the compiler flags and use r9 to access the GOT? With this in mind, there are a few options to have function calls to shared library functions working:

1. Patch ARM Embedded GCC
2. Rewrite PLT entries to use PIC base addressing
3. Bypass the PLT and jump directly to the function address held in a GOT entry

While patching GCC is the most direct way to fix this problem, it is out of scope for this particular project. However, Section 5.1 discusses potential patches to GCC in more depth.

Rewriting the code in the PLT entries is a promising option. Simply replace the existing PC-relative PLT code with instructions that will look up into the GOT using the PIC base register. However, this seems simpler than it is in reality. How does one locate the instructions to rewrite? Are they just any instructions that match exactly what GCC generates in the `.plt` section? It's possible that the new instructions take up more space and need to shift around other parts of the binary. Also, the compiled code is using an instruction set called *Thumb-2* which is a compressed version of the ARM architecture where each instruction can be either 16 or 32 bits wide, further complicating any binary parsing. In any case, binary writing is a whole topic in academia on its own with its own set of complexities [61, 51].

Finally, bypassing the PLT is what this implementation opted for. While the PLT is valuable for invoking the dynamic linker to resolve symbols at runtime, a dynamic linker is not needed to populate the GOT entry for the function address. As per Section 3.3, any function relocations will be handled by the flashing tool. This means, as long as the right GOT entry is populated with the function address, application code can jump directly to the address in that GOT entry when making a shared library call.

To bypass the PLT, there is a GCC code generation flag which meet the criteria. The flag is called `-fno-plt` and the GCC documentation describes it as such: “Do not use the PLT for external function calls in position-independent code. Instead, load the callee address at call sites from the GOT and branch to it.” [5]. While this flag appears to achieve the intended goal of directly accessing function addresses via the GOT, inspection of the generated code shows no difference in code generation. With the flag enabled on `arm-none-eabi-gcc` version 14.2.1, the PLT is still generated, and the code makes jumps to the PLT. Using tools like `diff` to compare file contents, shows no difference in the output binary file with or without the `-fno-plt` option [9]. Another post on the GNU ARM Embedded Q&A, titled “Option `-fno-plt` has no effect”, experiences the same issue with the `-fno-plt` flag [31]. One of the replies state that “Support for `-fno-plt` is not implemented presently”. However, another reply of the same post mentions an alternative. They state that “The use and creation of the plt can be circumvented by the use of function pointers. This will create a jump directly through GOT instead of creating a plt entry” [31].

Using function pointers certainly gets around using the PLT for function calls. Instead, the code loads the function pointer just as it would any other data member, dereferences the pointer and jumps to the resulting address. Figure 3.11 shows the difference in C code when declaring a function signature as a function pointer and Figure 3.14 shows the difference in generated assembly when making a function call via a function pointer. The main downside to using a function pointer is that it requires all shared library function declarations to be modified. These are typically found in the header files of libraries, so if a function pointer only header file was written for a library, it is possible that only the reference to the header could be swapped out with no changes to the rest of the actual application code.

```
int lib_func(int x);                int (*lib_func)(int x);
```

Figure 3.9. Traditional Function Declaration **Figure 3.10.** Function Pointer Declaration

Figure 3.11. Declaring a function (left) compared to a function pointer (right) in C

```

main:
; Set argument 1 to 23
movs    r0, #23
; Jump to PLT entry for lib_func
b.w     80001730

```

Figure 3.12. Calling a function without using function pointer which results in a jump to a PLT entry

```

main:
; Load function pointer GOT offset
ldr     r3, [pc, #8]
; Load GOT entry using PIC Base
; register:
; GOT Base (r9) + offset (r3)
ldr.w   r3, [r9, r3]
; Set argument 1 to 23
movs    r0, #23
; Dereference function pointer
ldr     r3, [r3, #0]
; Jump directly to lib_func
bx      r3

```

Figure 3.13. Calling a function with a function pointer. Accesses value of function pointer via GOT, dereferences pointer and makes the jump.

Figure 3.14. Generated assembly when calling a function with the PLT (left) and bypassing the PLT via a function pointer (right). This code was compiled from the application C code in figure 3.1

3.2 elf2tab

As described in Section 1.5.3, `elf2tab` is a tool which converts compiled ELF binaries into TAB files. In this process, it parses the ELF binary, includes only sections which are necessary, and packages this with a TBF Header to create a TBF file. In the context of shared libraries, `elf2tab` is useful for collecting relocation information needed to correctly map shared library function calls to the right address. While `elf2tab` is able to parse the contents of the ELF file, it has no knowledge of where application and shared library code will actually exist. However, Tockloader possesses this information since it chooses where to flash applications and shared libraries on the actual storage of the board. As a result, `elf2tab` can prepare the information about function relocations and let Tockloader perform the actual relocation.

For each function call to a shared library, `elf2tab` will produce a TOML file with relocation information to aid Tockloader. See Figure 3.15 for a sample TOML file that `elf2tab`

generates. Recall from Section 1.5.3, how `elf2tab` generates a TAB file which is really just a tar archive. `elf2tab` can just add the relocation TOML file to the archive alongside the other files. The TOML file contains a section for each function (e.g. `[lib_func]`). Within each section, it specifies the field “`app_name`” which is the name of the application making a call to a shared library function. The field “`app_got_addr`” represents the location of the GOT entry in the application binary. Specifically it is the address within the application TBF file created by `elf2tab`. This address is slightly different than where the GOT entry is located in the original ELF file because `elf2tab` changes the layout and positioning of the sections. Once the value at this location in the binary is patched, the correct value will be carried over once the `.got` section is loaded into memory at application startup. The field “`lib_name`” indicates the Tock package name of the shared library which contains the function being called. Finally, the “`lib_func_offset`” field represents the location of the function in the shared library. Note that this isn’t an offset into the shared library ELF file but the address that the linker assigned to the function symbol at link-time. The address starts with `0x8000_____` because the linker script places flash sections starting from `0x80000000`.

```
[lib_func]
app_name = "shlib_app"
app_got_addr = 0x1f20
lib_name = "libtest"
lib_func_offset = 0x80000029
```

Figure 3.15. `elf2tab` Relocation TOML File with a single entry for the function `lib_func`

To gather this relocation information, `elf2tab` looked through the dynamic symbol table of the application ELF file. The dynamic symbol table is a subset of the main symbol which only contains entries that concern the dynamic linker [15]. It searched the dynamic symbol table for any global function symbols. Once a list of functions needing relocations was collected, `elf2tab` inspected the relocation sections to find a relocation entry with the same function name. With the corresponding relocation entry acquired, `elf2tab` knows where the function’s GOT

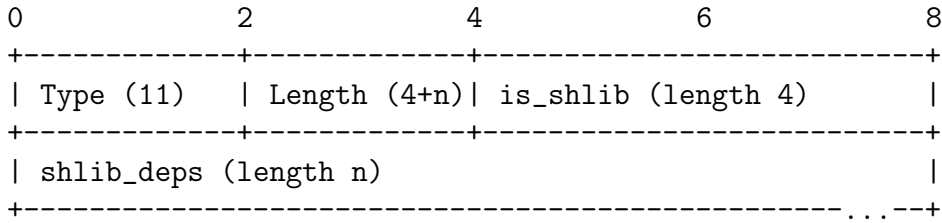


Figure 3.16. Shared Library TBF Header shown in Type-Length-Value (TLV) format.

entry is located and which part of the binary needs relocation. Finally, `elf2tab` searches for the function symbol within the shared library ELF file to find the location in the shared library’s text section. `elf2tab` is able to do this the path to any shared library dependencies are passed in via a new command line argument.

Finally, `elf2tab` added a new header to the TBF. This new header describes information about shared libraries. Firstly, there is a field which describes whether of not this TBF is a shared library or not. This is encoded as either 0 or 1. Secondly, it contains the package names of any shared libraries this TBF depends on. This is represented as a sequence of null-terminated strings. See Figure 3.16 for a block diagram.

3.3 Tockloader

Since Tock currently has no mechanism to dynamically create processes at runtime, Tockloader knows exactly which applications are running on a board at the time when applications are flashed. This means that Tockloader can perform any relocations needed for shared library function calls since it knows where both applications and shared libraries are located in flash memory. Therefore, relocations done by Tockloader incur no runtime cost since they occur when applications are being flashed, not at runtime.

Now that `elf2tab` has collected the relocation information, Tockloader can execute the actual relocation. Before flashing the application, the shared library dependency must be flashed with Tockloader. Thanks to the shared library TBF header (Section 3.2), Tockloader knows whether of not a TBF is a shared library.

```

int (*lib_func)(int x);

int main(void) {
    int result = lib_func(23);
    return result;
}

```

Figure 3.17. Application code from Figure 3.1 except modified to have a declaration of `lib_func` as a function pointer as per Section 3.1.3.

```

800001e4 <main>:
800001e4:      4b02          ldr     r3, [pc, #8]      @ (800001f0)
800001e6:      f859 3003    ldr.w   r3, [r9, r3]
800001ea:      2017          movs    r0, #23
800001ec:      681b          ldr     r3, [r3, #0]
800001ee:      4718          bx      r3
800001f0:      00000050      andeq   r0, r0, r0, asr r0

```

Figure 3.18. Disassembly of application code from Figure 3.17. Assume `r9` points to the start of the GOT (0x00000800 in the ELF file address space or at runtime: 0x00000800 + application memory start). Note that line at address 800001e4 is loading the constant from address 800001f0. This constant represents the offset into the GOT for the `lib_func` function pointer.

00000800 <_got>:	00000800 <_got>:
...	...
850: 00000000	850: 00044055
...	...

Figure 3.19. This figure shows how Tockloader will modify the GOT entry which should contain the location of `lib_func`. Left shows the GOT from an objdump of the compiled application ELF file. The binary was compiled from application code in Figure 3.17. Note how the disassembled code uses the offset 00000050 to access the entry for `lib_func`. The right side shows the modified binary once Tockloader has written the true address of the function `lib_func`. Note that this address points to flash memory. Once the GOT is loaded into memory, the application can freely use it to access and call into `lib_func`.

Once the shared library is flashed, the application TBF can be flashed. Tockloader reads the relocation TOML file provided by `elf2tab` and modifies the application binary before flashing it. Tockloader starts by reading the applications and shared libraries already flashed on the board's flash storage. Once Tockloader finds the shared library specified as a dependency in the

TOML file, it notes its location in flash. Then, Tockloader modifies the application binary before flashing it onto the board. It knows exactly where the GOT entry is in the executable file, so it has no need to perform additional ELF parsing. It just writes the location of the shared library function to the application's corresponding GOT entry. Figure 3.19 shows how Tockloader modifies the GOT in the application binary and Figure 3.18 shows how the compiled code uses that modified GOT entry.

With the function pointer workaround discussed in Section 3.1.3 and shown in figure 3.14, Tockloader must add an extra level of indirection when performing the relocation. When calling a function via a function pointer, the code the compiler emits will first try to dereference the function pointer before jumping to the resulting address. This means that Tockloader cannot simply write the address of the function into the GOT entry. Instead, it must write the address of the function into another location and then write the address of that location into the GOT entry. This way, when the application dereferences the function pointer from the GOT, it will get the correct function address to jump to. In the implementation, Tockloader appends these function addresses to the end of the application binary and writes the address of these appended locations into the GOT entries. Figure 3.20 shows a comparison of how Tockloader performs relocation with and without function pointers and how function pointers are laid out in the binary.

Figure 3.21 shows the full installation process and dependency graph for flashing a shared library and dependent application onto a board.

3.4 Tock Kernel

Shared libraries were chosen to closely match the format and layout of Tock applications. They already use the same TBF format, so the kernel can parse them and load them just the same as any other Tock application. When the kernel is loading shared libraries and applications from flash, the main difference is that it doesn't schedule shared libraries to be scheduled like normal applications are. To determine which entities are applications or shared libraries, it checks the

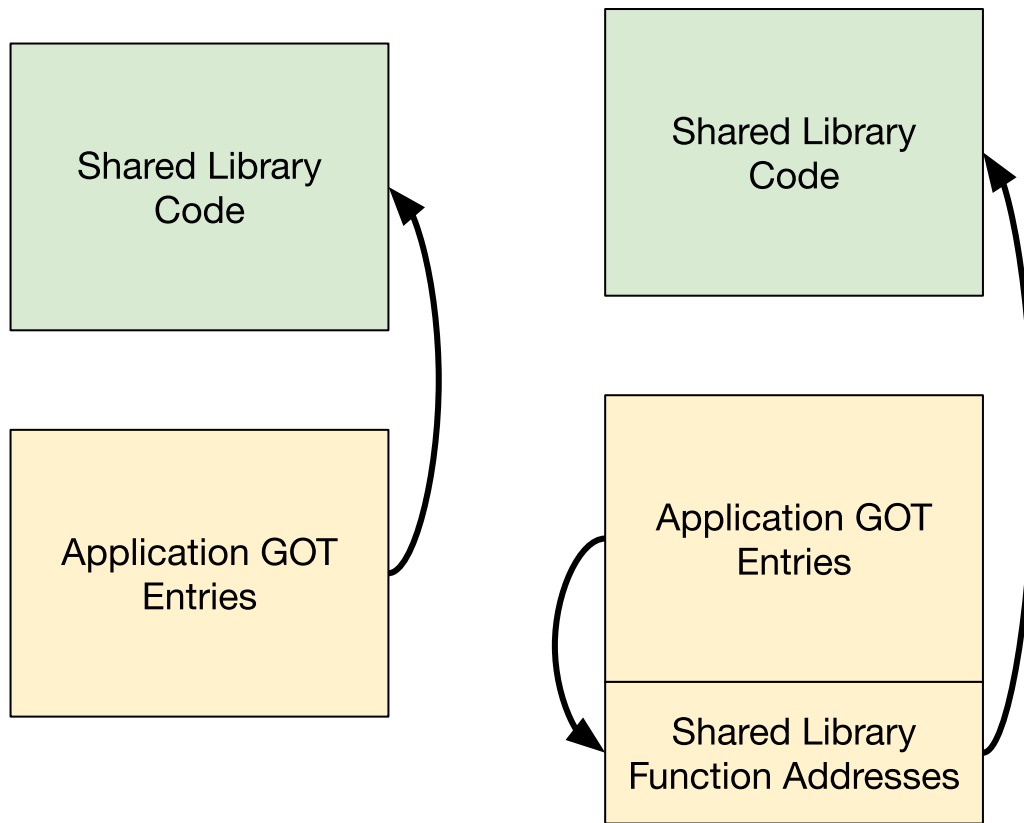


Figure 3.20. The left shows how Tockloader performs relocation when the application calls a shared library function directly (without function pointers). The right shows how Tockloader performs relocation when the application uses function pointers to call shared library functions. Note how an extra level of indirection is needed when using function pointers. Green represents the shared library while yellow represents the application binary.

shared library header of the TBF (see Section 3.2 and Figure 3.16).

Once the kernel loads an application which depends on a shared library, it allocates enough memory for the data sections of the shared library on top of the memory requested by the main application. The kernel also sets up the MPU accordingly with which shared library an application needs. When an application is executing, it will have an MPU region allocated for every shared library it is using. This MPU region will have read and execute permissions to allow the application to jump into the sections of flash that contain the shared library text sections. When a context switch to another application occurs, Tock will already swap out MPU regions.

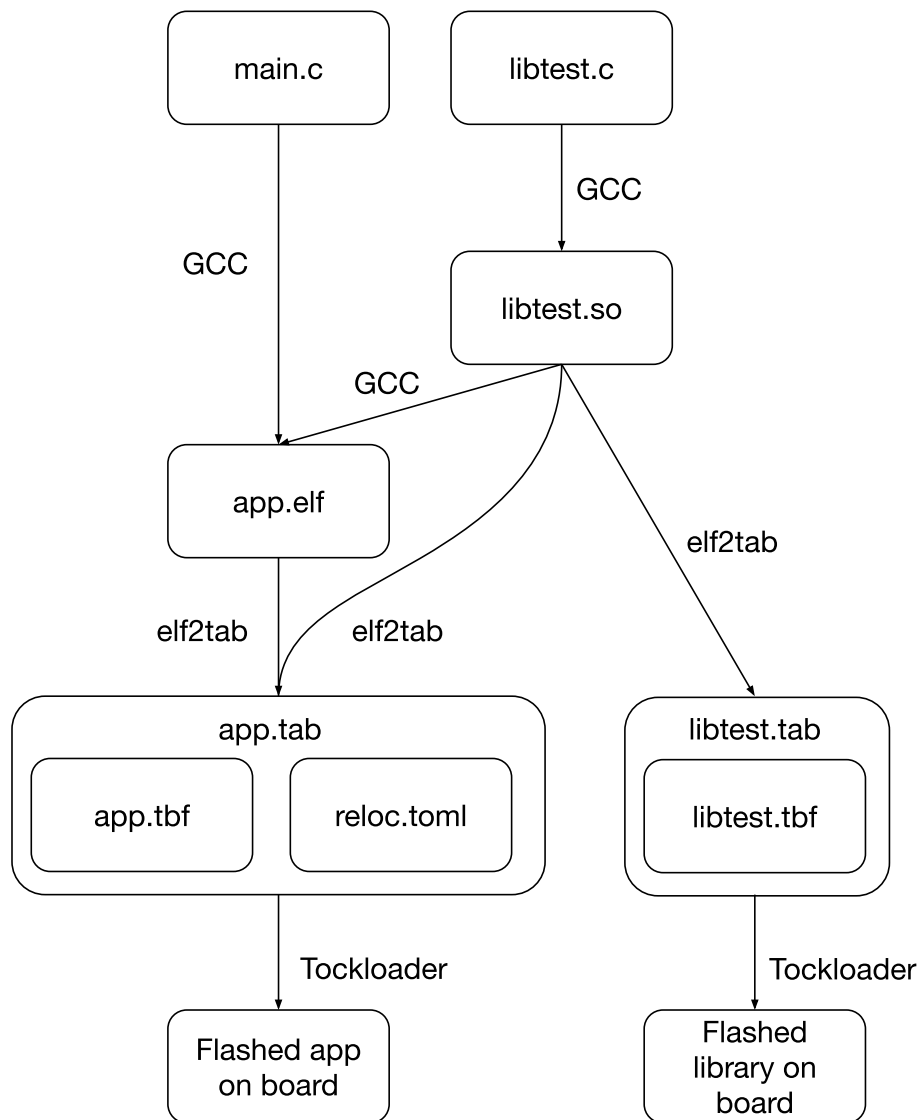


Figure 3.21. Dependency Graph of Installing Shared Libraries and Applications

3.5 libtock-c

Finally, the application startup (`crt0`) routine needed to be updated to support shared libraries. Recall from Section 1.5.2, how `libtock-c` includes code which sets up the stack, PIC base register, GOT, data sections, and BSS sections. `crt0` needed to be modified to perform required setup for shared libraries. Just like for applications, `crt0` needs to copy over the shared library GOT sections from the binary in flash to memory. In addition, it sets the shared library's separate PIC base register (e.g. `r10`) as outlined in Section 3.1.1. Since it just loaded where the shared library GOT is in memory, it knows exactly what to set the PIC base register to. Finally, `crt0` copies over the data sections from flash and initializes BSS sections. To know where the shared library sections exist in flash, `crt0` will make a system call to the kernel to find out the location of where shared library dependencies exist in flash memory. Then, `crt0` will read the first few bytes of the shared library in flash to read the shared library's `crt0_header`. Recall from Section 1.5.2, where the `crt0_header` is a header created by the linker script which is placed at the start of an application binary. The linker script for shared libraries also creates a `crt0_header` which contains locations of sections such as the GOT, data and BSS within the binary. Knowledge of these locations is useful for `crt0` as it can know exactly where to fetch sections from the app binary in flash. The system call to gain knowledge of the shared library's location in flash hasn't been implemented yet, but once it is, `crt0` would not need to change much. It already reads the shared library's `crt0_header`, it would just need to learn of the shared library's location via the syscall instead of being manually informed.

3.6 Walkthrough

3.6.1 Overview

The walkthrough will follow along the dependency chain in Figure 3.21. Each step will produce an artifact needed for the next step. The end result will be a shared library and an application which uses that shared library, both flashed onto a Nordic Semiconductor nRF52840

Development Kit board.

3.6.2 Environment

Hardware

This walkthrough assumes the use of a Nordic Semiconductor nRF52840 Development Kit (DK) board [27]. This board is popular in the Tock community and is well supported by Tock. It has an ARM Cortex-M4 CPU which is compatible with the ARMv7-M architecture [4].

Other ARM boards can be easily supported. They simply need their board-specific files in the Tock kernel to be updated. The changes should be similar to the ones found in `boards/nordic/nrf52840dk/src/lib.rs` and `boards/nordic/nrf52840dk/src/main.rs` from the latest commit of the `shared-libraries` branch on the Tock kernel fork linked below.

Software

This walkthrough was tested on Ubuntu 24.04 and NixOS 25.05. Other platforms (i.e. macOS and Windows) are supported. See the respective README of each repository for platform specific setup instructions.

3.6.3 Clone

The modifications applied to the aforementioned Tock projects can be found in several GitHub repositories. On each repository, the `shared-libraries` branch contains the modifications needed for shared library support. The following table provides links to the `shared-libraries` branch for each of the projects.

Use Git to clone each of the repositories and **checkout the `shared-libraries` branch**.

For example, the following command will clone `libtock-c` and checkout the

Table 3.1. Links to repositories

Name	Repository
libtock-c	https://github.com/atar13/libtock-c/tree/shared-libraries
elf2tab	https://github.com/atar13/elf2tab/tree/shared-libraries
Tockloader	https://github.com/atar13/tockloader/tree/shared-libraries
Tock Kernel	https://github.com/atar13/tock/tree/shared-libraries

shared-libraries branch:

```
git clone -b shared-libraries https://github.com/atar13/libtock-c.git
```

Alternatively, running `git checkout shared-libraries` for each of the projects will check out to the branch with the necessary changes.

The walkthrough assumes the following directory structure where each project is adjacent to each other:

```
.
|-- elf2tab
|-- libtock-c
|-- tock
|-- tockloader
```

3.6.4 Dependencies

Install the appropriate toolchains and necessary tools for each of the projects. Each project has a `README.md` file which outlines the dependencies needed. Here is a page which outlines many of the necessary tools for working with Tock:

github.com/tock/tock/blob/master/doc/Getting_Started.md

Here are a few important dependencies that will be needed:

1. **ARM GCC** is used to build the userspace C shared library and application. On Ubuntu it can be installed via `sudo apt install gcc-arm-none-eabi`. See the `libtock-c` `README` for more information.

2. **make** is also required for building the userspace shared library and application.
3. **cargo & rustc** are needed to build `elf2tab` and the kernel. See <https://rust-lang.org/tools/install/> for getting started with building Rust.
4. **Python & pip** are used to run Tockloader and install its dependencies. Install them from <https://www.python.org/downloads/>.
5. **tockloader** will be used to flash the kernel, apps and shared libraries. While a patched version of `tockloader` from the `shared-libraries` branch will be needed to flash shared libraries and applications, the standard release can be used to flash the kernel. This can be installed via `pipx` with `pipx install tockloader`.
6. **A Programming Adapter** is likely needed by `tockloader` to actually perform the flashing operation. See this page for more details: <https://github.com/tock/tock/blob/master/doc/Getting-Started.md#programming-adapter>.

Nix

If you're using the Nix package manager, you can use the provided `shell.nix` and `flake.nix` files in the root of each repository to enter a shell with the necessary dependencies. For example, to enter a shell with the dependencies for `libtock-c`, run `nix-shell` in the root of the `libtock-c` repository. If you're familiar with Nix flakes, `tockloader` and `elf2tab` have a `flake.nix` file which can be used to enter a shell with pinned versions of dependencies via running `nix develop` in the root of the repository.

3.6.5 Build and Flash Tock Kernel

Clone the fork of the Tock kernel from github.com/atar13/tock/tree/shared-libraries and set your current directory as the root directory of the project.

Run the following commands to build the modified Tock kernel and flash it onto the Nordic nRF52840DK board. The `make flash` command assumes you have `tockloader` available in

your shell. Flashing the kernel doesn't require a modified version of `tockloader` unlike when flashing applications and shared libraries (See Section 3.6.4 on how to get `tockloader`).

```
cd boards/nordic/nrf52840dk
make flash
```

If you run into permission issues when flashing the board, it's likely that you don't have permission to access your serial port. On Linux, make sure your user is a member of the `dialout` group.

Once the kernel is flashed, run `tockloader listen` to verify it is running. If the kernel is up and running, you should see the following console prompt. Type `help` to see the available commands.

```
tock$
```

3.6.6 Build a shared library

Clone the fork of the `libtock-c` from github.com/atar13/libtock-c/tree/shared-libraries and set your current directory as the root directory of the project.

Compile the sample shared library to produce `libtest.so` (which can be found in `libtest/build/cortex-m4/libtest` after building).

```
cd libtest
make
```

The source code of the shared library is relatively simple and contains one function (`lib_func`) which is defined as such (found at `libtest/lib.c`):

3.6.7 Build an application

We will compile the sample application which makes a call to our shared library function (`lib_func`) and prints the result.

```

int glob = 42;
int lib_func(int x) {
    return x + glob;
}

```

Figure 3.22. Sample Shared Library C Code

Set your current directory to the root of the `libtock_c` repository and run the following commands.

```

cd examples/shlib_app
make

```

Here is the source code of the sample application(found at `examples/shlib_app/main.c`). To keep this demo simple, the `printf` and `libtocksync_alarm_delay_ms` functions are statically linked into the application binary.

```

int app_glob = 10;
int main(void) {
    while (1) {
        int result = lib_func(app_glob);
        printf("lib_func(%d) = %d\n", app_glob, result);
        libtocksync_alarm_delay_ms(1000);
    }
}

```

Figure 3.23. Sample Application C Code

3.6.8 Create shared library TAB file

Clone the fork of the `libtock-c` from github.com/atar13/libtock-c/tree/shared-libraries and set your current directory as the root directory of the project.

The following commands build the modified version of `elf2tab`.

```

cargo build --release

```

Now, use it to create a TAB file from the shared library ELF file that was built earlier. Make sure to update the path to the `libtest.so` file. The provided command assumes `libtest.so` is in the build directory of `libtock-c/libtest` and that `libtock-c` is adjacent to the `elf2tab` folder. Alternatively, the same command is provided in the script: `./elf2tab_shared_library.sh`.

```
./target/release/elf2tab -n libtest --stack 2048 --app-heap 1024
--kernel-heap 1024 --kernel-major 2 --kernel-minor 0
--minimum-footer-size 3000 -v -o libtest.tab
../libtock-c/libtest/build/cortex-m4/libtest.so
```

3.6.9 Create application TAB file

Like the previous step, `elf2tab` will generate a TAB file from the application ELF file. Note that this command requires the `--shared_library_deps` option to point to the shared library binary (`.so` file). Alternatively, the same command is provided in the script:

`./elf2tab_application.sh`.

```
./target/release/elf2tab -n shlib_app --stack 2048 --app-heap 1024
--kernel-heap 1024 --kernel-major 2 --kernel-minor 0
--minimum-footer-size 3000 --shared_library_deps
../libtock-c/libtest/build/cortex-m4/libtest.so
-v -o shlib_app.tab
../libtock-c/examples/shlib_app/build/cortex-m4/cortex-m4.elf
```

3.6.10 Flash Shared Library to Board

Clone the fork of `tockloader` from github.com/atar13/tockloader/tree/shared-libraries and set your current directory as the root directory of the project. Be sure that you use this modified version of `tockloader` and not the standard release of `tockloader` which may be in your shell's `PATH` after installing with `pip/pipx`.

Like the commands before, the following command assumes your clone of `tockloader` is adjacent to `elf2tab`.

```
python3 -m tockloader.main install ../elf2tab/libtest.tab
```

Verify the shared library has been flashed by running:

```
python3 -m tockloader.main list
```

There should be an entry for the shared library that looks like Figure 3.24.

Shared Library 0	
Name:	libtest
Version:	0
Enabled:	True
Sticky:	False
Total Size in Flash:	4096 bytes

Figure 3.24. Shared library in tockloader list

3.6.11 Flash application to board

Similar to the previous step, we will use tockloader to flash the application which depends on the shared library.

```
python3 -m tockloader.main install ../elf2tab/shlib_app.tab
```

Again, verify the application has been flashed by running:

```
python3 -m tockloader.main list
```

There should be an entry for the application that looks like Figure 3.25.

3.6.12 Verify application output

Finally, use tockloader to connect to the board's serial output and verify the application is running and correctly calling into the shared library.

App 0	
Name:	shlib_app
Version:	0
Enabled:	True
Sticky:	False
Total Size in Flash:	16384 bytes

Figure 3.25. Application in tockloader list

```
python3 -m tockloader.main listen
```

You should see output similar to this:

```
lib_func(10) = 52
lib_func(10) = 52
lib_func(10) = 52
...
```

Recall from Figure 3.22 that the shared library function `lib_func` adds 42 to its argument. Since the application is passing in 10, the expected result is 52.

Feel free to modify the application and shared library code to experiment with different function calls and arguments. Just remember to rebuild the shared library and application, create new TAB files, and reflash them onto the board.

Chapter 4

Evaluation

As outlined in the design goals of Section 2, using shared libraries should not incur a high runtime cost. I considered three types of computation where shared libraries introduce extra overhead: function resolution at flash time, application startup, and calls to shared library functions.

Traditionally, desktop and server class systems resolve references to shared library functions at runtime via a dynamic linker. The dynamic linker is invoked either when the process is loaded or the first time a function is called with lazy loading (see Section 1.3.2). As discussed in Section 3.3, Tockloader acts as a dynamic linker and performs resolution for shared library functions. Since Tockloader operates at “flash-time”, it performs these relocations before the board ever runs the application. Therefore, the overhead of Tockloader’s relocations will not be quantified as part of “runtime”.

With regard to application startup, the `crt0` routine was updated to perform necessary setup for shared libraries as outlined in Section 3.5. As a reminder, `crt0` is the entry point where application code begins executing before entering the `main` function. For both applications and shared libraries, `crt0` is responsible for copying GOT and data sections from the binary to memory, zeroing the BSS section, and setting the respective PIC base registers. I measured the overhead introduced by initializing a shared library at startup using the cycle counter provided by the ARM Data Watchpoint and Trace unit (DWT) [8]. For initializing the simple shared library

introduced in Figure 3.2, initialization took 391 cycles. Table 4.1 compares the major tasks that `crt0` performs to set up a shared library. However, most of this work isn't additional overhead, but would have been performed without shared libraries. If the same library was statically linked instead of dynamically linked as a shared library, `crt0` would perform the same setup steps (besides setting the shared library PIC base register). The only difference would be that the sections would be loaded from the main application binary rather than from a separate shared library. So, the main additional overhead which didn't exist before is simply setting the shared library PIC base register which only takes a handful of cycles once at startup.

Table 4.1. Comparison of various initialization steps that occur during `crt0` to prepare a shared library to be executed.

	Cycles
Load GOT	178
Load Data	108
Zero BSS	26
Set PIC base register	8

The overhead incurred by shared library function calls is more critical to minimize. During the lifetime of an application, a shared library function call can happen hundreds or thousands of times. Additionally, if a library call ends up on the hot-path of computation it is crucial that the library call is as efficient as possible.

As outlined in Section 3.1.1, the current implementation sets aside another register (e.g. `r10`) to use as the shared library PIC base and also requires all shared library functions to be declared as function pointers. Figure 4.1 compares the ARM assembly instructions needed for an application to perform a traditional single argument function call as opposed to a single argument shared library function call with the current implementation. In the traditional case, it takes a total of two instructions to make the function call: a single instruction to set the first argument and another to jump to the function code. In the shared library case, a total of five instructions are needed. One instruction is needed to load the function pointer's offset within the GOT to register `r3`. Next, the GOT lookup is performed with the application's PIC base register. The

		ldr	r3, [pc, #48]
movs	r0, #23	ldr.w	r6, [r9, r3]
bl	800001e4 <my_func>	ldr	r3, [r6, #0]
		movs	r0, #23
		blx	r3

Figure 4.1. The left side depicts the ARM assembly needed to perform a traditional function call with a single argument. The right side shows the assembly needed to perform a function call to a shared library (which includes a GOT lookup and a function pointer dereference). Both examples are calling a function with the same function signature from Figure 3.2

Table 4.2. Single Argument Shared Library Function Call Overhead. Baseline is a traditional direct function call without shared libraries (see left side of Figure 4.1). **Function Pointer** is the current implementation which calls a function pointer via a GOT lookup (see right side of Figure 4.1). No FP is an alternative scenario which looks up the function address directly from the GOT. The last case “Save/Restore r9 w FP” builds on top of the current **Function Pointer** approach and adds instructions to save and restore the application PIC base register. See Section 3.1.1 for a discussion on why this saving and restoring functionality is needed.

	Baseline	Function Pointer (FP)	No FP	Save/Restore r9 w FP
Instruction Count	2	5	4	7

function pointer address is loaded into register r6. Then, the function pointer is dereferenced and the function’s actual address is placed in register r3. The remaining two instructions closely model the original case: one sets the first argument and the other jumps to the function code. In short, the current implementation of shared libraries incurs an overhead of three additional instructions which are needed to look up the location of the function at runtime.

If the implementation was modified to not use function pointers, one instruction could be shaved off. Instead of dealing with function pointer, the entry in the GOT would just contain the address of the function itself. This would remove the need to dereference the function pointer before jumping.

Another variation discussed in Section 3.1.1, involves saving and restoring the special PIC base register r9 on every function call. As discussed earlier, this version would share a single PIC base register between applications and shared libraries as opposed to the current version which uses separate PIC base registers (r9 and r10). With regard to runtime, this would add additional overhead to save and restore r9 on every function call. While there’s no compiler

option to perform this operation on every shared library function call, the pseudo ARM assembly from Figure 3.6 would suggest only two additional instructions: one to save r9 and one to restore it after the library function returns. Table 4.2 compares the instruction overhead of a few of these variations.

Chapter 5

Discussion and Future Work

5.1 Compiler Patches

This section discusses potential extensions to the ARM port of GCC to aid in smoothing the edges around shared libraries on Tock. To avoid conflicts with other platforms, these flags could be marked explicitly for the Tock platform.

5.1.1 Avoid clobbering PIC base register

As discussed in Section 3.1.1, Tock applications expect the register `r9` to be reserved as a special *PIC base* register which always stores the base address of an application's GOT in memory. Since shared library's have their own GOT in memory, separate from the main application GOT, they need their own way to access their GOT without using PC-relative memory instructions. The solution proposed for this implementation is to reserve another register (e.g. `r10`) as the PIC base register for a shared library. However, this isn't the ideal solution since it severely restricts the number of shared libraries that can be used at a time. There's only a handful of registers where many are already taken. Even worse, compiling shared libraries for a specific PIC base register makes the shared library code harder to share. If multiple apps are using the same shared library, each app needs to make sure that they aren't using another shared library which is compiled with the same PIC base register. If two shared libraries are compiled for the same PIC base (e.g. `r10`) then one of them would be incorrectly assuming that `r10` was pointing

to their own GOT.

As an alternative explored in Section 3.1.1, it would be possible to have the application and shared libraries share a single PIC base register. This would require additional code to save and restore the PIC base register on every call to a shared library function. Note that this would only occur on *shared library function calls* and not calls to functions within the main application executable. While there's currently no code generation compiler option for this, it could be added to GCC. It would introduce additional code on the caller side to save its PIC base register to the stack, make the function call, and restore the PIC base once the call returns. On the callee side, it loads its own PIC base register and stores it before the function returns. Figure 3.6 shows some pseudo ARM assembly of such functionality.

Saving and restoring the register is straightforward on the caller's side thanks to instructions that can pop and push the reg to the stack. However, the callee can't load and store their PIC base on the stack since their stack frame is created when the function is called and is destroyed after the function returns. Instead, the shared library's PIC base value must be stored elsewhere in memory. The actual location could be allocated by `crt0` and `crt0` could update references to the PIC base value when the shared library is loaded. Instead of updating every reference to the PIC base, the code required to store and load the PIC base could be reused as sub-routines, similar to how PLT entries work. When the shared library needs to load its PIC base register, it would call to a "load" sub-routine which would have the relocated address to the PIC base value and a similar sub-routine would exist for storing the PIC base back in memory. With this model, relocations only need to be applied once for the store and load sub-routines. However, this introduces a problem if multiple apps try to share the same library. Each app using a library needs a separate library GOT for each app using the library. This means the PIC base will be different for the same library being used across multiple apps. Therefore, it isn't feasible for `crt0` to directly modify the shared library code to be hard-coded with the location of a GOT. One alternative approach could be to have these load and store subroutines be in an executable region of memory where a different version of the sub-routine could exist for each application

using the library. With this all being said, the best design for such a compiler extension is yet to be discovered, and future work could explore trade-offs across different implementations.

5.1.2 PLT entries use PIC base register

The current implementation avoids using the PLT altogether due to issues in PLT generation with ARM for GCC as described in Section 3.1.3. The code in the PLT isn't generating the correct position-independent code which uses the PIC base register. The code generated in the PLT entries uses PC-relative addressing which is incompatible with the non-relative text and data sections present for Tock applications. The compiler patch to fix this issue would be relatively straightforward, change the PLT entries to access the GOT using the PIC base register when compiled with the `-mpic-register=r9`, just like all other code already does. Figure 5.1 compares PLT code currently generated by ARM GCC to the proposed version which uses the PIC base register.

<pre><i>; load PC-relative offset</i> <i>; to the GOT</i> movw ip, #62160 movt ip, #32767 <i>; calculate actual GOT addr</i> add ip, pc <i>; read GOT entry and jump</i> <i>; to function</i> ldr.w pc, [ip]</pre>	<pre><i>; load offset into GOT</i> ldr r3, [pc, #48] <i>; read GOT entry</i> ldr.w r3, [r9, r3] <i>; jump to function</i> bx r3</pre>
--	---

Figure 5.1. The left shows current PLT entries which uses PC-relative offsets to access the GOT in favor of the PIC base register. A hard-coded constant is offset from the current PC is used to lookup the function location cached in the GOT. Once the function location is found, it is loaded into the PC register to start executing that function. The right depicts what the PLT should look like when compiled to use the PIC base register. First it loads the offset within the GOT for the specific function being called. While this offset is loaded using PC-relative addressing, it's totally fine since it's located nearby in the text section (fixed offset) and the offset within the GOT will not change after compile-time. Next, the GOT entry is read using PIC base relative addressing ($r9 + \text{offset within GOT}$). Finally, it jumps to the actual function body.

Alternatively, a compiler patch could fix the functionality of the `-fno-plt` option to correctly avoid using the PLT and load function addresses directly from the GOT. See Section

3.1.1 for a detailed explanation of the flag’s functionality and how GCC for ARM doesn’t implement it. This compiler modification would allow function calls to jump directly to the function location stored in the GOT and avoid going through the PLT altogether.

5.2 RISC-V and Rust Userspace

The current model for shared libraries will not generalize to RISC-V boards and apps written in Rust due to lack of toolchain support for PIC. The RISC-V version of GCC doesn’t support the `-mno-pic-data-is-text-relative` flag discussed earlier [35, 26]. LLVM, the Rust compiler’s code generator, also doesn’t support the same embedded position-independent code features available on GCC [23, 10, 33].

Currently, Tock compiles RISC-V and Rust applications for several fixed addresses in both Flash and RAM [6]. This way, it can pick and choose which combination of fixed addresses can coexist on the board at runtime. It’s possible for shared libraries to also be compiled for several fixed addresses. Once one of the fixed addresses are chosen, relocations can be applied to point to their actual location.

5.3 Nested Shared Libraries Calls

The current implementation doesn’t allow shared libraries to make calls to other shared libraries. To support this, shared libraries could be linked with other shared libraries just like applications were in Section 3.1.1. Tockloader would need to be updated to perform relocations wherever shared libraries call other shared libraries.

5.4 Accessing shared library global data from application

Currently, applications cannot access global data defined in shared libraries. This is because the application doesn’t have a way to find the location of the shared library’s GOT in memory. One potential solution is to have `crt0` store the shared library’s PIC base register value

into a known location in memory after initializing the shared library. The application could then read this value from memory to find the shared library's GOT and access global data. However, a compiler modification would be needed to generate code which can access global data from a shared library using the shared library's PIC base register.

Chapter 6

Conclusion

This work presents an initial implementation that supports shared libraries for the Tock operating system. The current implementation supports function calls to shared libraries where the library can make its own global data accesses. The Tock kernel, `elf2tab`, Tockloader, application build system, and application startup routine have been modified to provide the needed infrastructure for shared libraries to function. While a working proof of concept exists, there are several rough edges that limit the number of shared libraries and require small modifications to source code. However, with the appropriate toolchain updates, the rough edges can be smoothed out and using shared libraries on Tock can be made to be seamless.

Bibliography

- [1] Address Space Layout Randomization (ASLR). https://en.wikipedia.org/wiki/Address_space_layout_randomization.
- [2] A.out - OSDev Wiki. <https://wiki.osdev.org/A.out>.
- [3] arm-none-eabi-gcc(1) — Arch manual pages. <https://man.archlinux.org/man/arm-none-eabi-gcc.1.en>.
- [4] ARMv7-M Architecture Reference Manual. <https://documentation-service.arm.com/static/5f8fef3af86e16515cdbf816>.
- [5] Code Gen Options (Using the GNU Compiler Collection (GCC)). <https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/Code-Gen-Options.html>.
- [6] Compilation - The Tock Book. <https://book.tockos.org/doc/compilation.html>.
- [7] Copy-on-Write. <https://open-education-hub.github.io/operating-systems/Lab/Compute/copy-on-write/>.
- [8] Data Watchpoint and Trace unit. <https://developer.arm.com/documentation/101051/0101/Data-Watchpoint-and-Trace-unit>.
- [9] diff(1) - Linux manual page. <https://www.man7.org/linux/man-pages/man1/diff.1.html>.
- [10] Does clang support to generate GOT-based position independent code with pic-register? - Clang Frontend / Using Clang. <https://discourse.llvm.org/t/does-clang-support-to-generate-got-based-position-independent-code-with-pic-register/64129>.
- [11] execve(2) - Linux manual page. <https://www.man7.org/linux/man-pages/man2/execve.2.html>.
- [12] Explicit Reg Vars - Using the GNU Compiler Collection (GCC). <https://gcc.gnu.org/onlinedocs/gcc-4.6.1/gcc/Explicit-Reg-Vars.html#Explicit-Reg-Vars>.
- [13] Fat binary. https://en.wikipedia.org/wiki/Fat_binary.
- [14] File:Elf-layout--en.svg - Wikimedia Commons. <https://commons.wikimedia.org/wiki/File:Elf-layout--en.svg>.

- [15] Introduction to the ELF Format (Part VI) : The Symbol Table and Relocations (Part 1). <https://blog.k3170makan.com/2018/10/introduction-to-elf-format-part-vi.html>.
- [16] J-Link. <https://www.segger.com/products/debug-probes/j-link/>.
- [17] Kernel Documentation - The Tock Book. <https://book.tockos.org/doc/>.
- [18] ld.so(8) - Linux manual page. <https://www.man7.org/linux/man-pages/man8/ld.so.8.html>.
- [19] Lecture 19: Shared libraries. <https://jhucsf.github.io/spring2021/lectures/lecture19-public.pdf>.
- [20] libtock-c Repository. <https://github.com/tock/libtock-c/>.
- [21] Link Options (Using the GNU Compiler Collection (GCC)). <https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/Link-Options.html>.
- [22] Linker Scripts - OSDev Wiki. https://wiki.osdev.org/Linker_Scripts.
- [23] [llvm-dev] [RFC][ARM] Add support for embedded position-independent code (ROPI/RWPI). <https://lists.llvm.org/pipermail/llvm-dev/2015-December/093022.html>.
- [24] MOVT — ARM Compiler toolchain Assembler Reference Version 4.1. <https://developer.arm.com/documentation/dui0489/c/arm-and-thumb-instructions/general-data-processing-instructions/movt>.
- [25] Needed to create own shared library in freertos / amazon freertos, any help is highly appreciated - FreeRTOS Community Forums. <https://forums.freertos.org/t/needed-to-create-own-shared-library-in-freertos-amazon-freertos-any-help-is-highly-appreciated/11959/4>.
- [26] No support for -mno-pic-data-is-text-relative -msingle-pic-base options? #1587. <https://github.com/riscv-collab/riscv-gnu-toolchain/issues/1587>.
- [27] nRF52840 Development Kit. <https://www.nordicsemi.com/Products/Development-hardware/nRF52840-DK>.
- [28] objdump(1) - Linux manual page. <https://www.man7.org/linux/man-pages/man1/objdump.1.html>.
- [29] OpenOCD. <https://openocd.org/>.
- [30] Program Header. <https://www.sco.com/developers/gabi/latest/ch5.pheader.html>.
- [31] Question #669758 “Option -fno-plt has no effect” : Questions : GNU Arm Embedded Toolchain. <https://answers.launchpad.net/gcc-arm-embedded/+question/669758>.
- [32] Question #675869 “-mno-pic-data-is-text-relative does not create...” : Questions : GNU Arm Embedded Toolchain. <https://answers.launchpad.net/gcc-arm-embedded/+question/675869>.

- [33] [RFC][ARM] Add support for embedded position-independent code (ROPI/RWPI). <https://discourse.llvm.org/t/rfc-arm-add-support-for-embedded-position-independent-code-ropi-rwpi/40323/>.
- [34] Shared object files or dynamic libraries #68782 . <https://github.com/zephyrproject-rtos/zephyr/discussions/68782>.
- [35] Support for -mno-pic-data-is-text-relative? #158. <https://github.com/riscvarchive/riscv-gcc/issues/158>.
- [36] tar(1) - Linux manual page. <https://www.man7.org/linux/man-pages/man1/tar.1.html>.
- [37] Tock Binary Format - The Tock Book. https://book.tockos.org/doc/tock_binary_format.
- [38] Type-length-value. <https://en.wikipedia.org/wiki/Type%E2%80%93length%E2%80%93value>.
- [39] What is GNU/Linux? — get GNU/Linux! <https://getgnulinux.org/en/linux/>.
- [40] Dynamic Linking - FreeRTOS. https://www.freertos.org/FreeRTOS_Support_Forum_Archive/October_2011/freertos_Dynamic_Linking_4761511.html, October 2011.
- [41] Memory Protection Unit (MPU). <https://documentation-service.arm.com/static/5ef61f08dbdee951c1ccdd48>, 2016.
- [42] Mastering the GNU linker script. <https://allthingsembedded.com/post/2020-04-11-mastering-the-gnu-linker-script/>, April 2020.
- [43] Daroc Alden. A look at dynamic linking [LWN.net]. <https://lwn.net/Articles/961117/>, February 2024.
- [44] Hudson Ayers, Evan Laufer, Paul Mure, Jaehyeon Park, Eduardo Rodelo, Thea Rossman, Andrey Pronin, Philip Levis, and Johnathan Van Why. Tighten rust’s belt: shrinking embedded rust binaries. In *Proceedings of the 23rd ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*, LCTES 2022, page 121–132, New York, NY, USA, 2022. Association for Computing Machinery.
- [45] Eli Bendersky. Load-time relocation of shared libraries . <https://eli.thegreenplace.net/2011/08/25/load-time-relocation-of-shared-libraries>, August 2011.
- [46] Eli Bendersky. Position Independent Code (PIC) in shared libraries. <https://eli.thegreenplace.net/2011/11/03/position-independent-code-pic-in-shared-libraries/>, November 2011.
- [47] Randal E. Bryant and David R. O’Hallaron. *Computer Systems: A Programmer’s Perspective*. 3/e (cs:app3e) edition, 2016.

- [48] Tool Interface Standard (TIS) Committee. Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification Version 1.2. <https://refspecs.linuxfoundation.org/elf/elf.pdf>, May 1995.
- [49] David Drysdale. How programs get run: ELF binaries [LWN.net]. <https://lwn.net/Articles/631631/>, February 2015.
- [50] David Drysdale. How programs get run [LWN.net]. <https://lwn.net/Articles/630727/>, January 2015.
- [51] Dongsoo Ha, Wenhui Jin, and Heekuck Oh. Repica: Rewriting position independent code of arm. *IEEE Access*, 6:50488–50509, 2018.
- [52] Aditya Kumar. Code size compiler optimizations and techniques for embedded systems. <https://llvm.org/devmtg/2020-09/slides/Kumar-LLVMDevCodeSizePdf.pdf>.
- [53] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B. Giffin, Pat Pannuto, Prabal Dutta, and Philip Levis. Multiprogramming a 64kb computer safely and efficiently. In *Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17*, page 234–251, New York, NY, USA, 2017. Association for Computing Machinery.
- [54] Victor Malumbres, Jose Saldana, Gonzalo Berné, and Julio Modrego. Firmware updates over the air via lora: Unicast and broadcast combination for boosting update speed. *Sensors*, 24(7), 2024.
- [55] Jonathan McGee. C/C++ Runtime Startup. <https://etherealwake.com/2021/09/crt-startup/>, September 2021.
- [56] Huy Dat Nguyen, Nicolas Le Sommer, and Yves Mahéo. Over-the-air firmware update in lorawan networks: A new module-based approach. *Procedia Computer Science*, 241:154–161, 2024. 19th International Conference on Future Networks and Communications/ 21th International Conference on Mobile Systems and Pervasive Computing/14th International Conference on Sustainable Energy Information Technology.
- [57] Weizhong Qiang, Yong Cao, Weiqi Dai, Deqing Zou, Hai Jin, and Benxi Liu. Libsec: A hardware virtualization-based isolation for shared library. In *2017 IEEE 19th International Conference on High Performance Computing and Communications; IEEE 15th International Conference on Smart City; IEEE 3rd International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, pages 34–41, 2017.
- [58] Amir Rachum. Shared Libraries: Understanding Dynamic Loading. <https://amir.rachum.com/shared-libraries/>, September 2016.
- [59] Erich Styger. Position-Independent Code with GCC for ARM Cortex-M. <https://mcuoneclipse.com/2021/06/05/position-independent-code-with-gcc-for-arm-cortex-m/>, June 2021.

- [60] Scott Thornton. Zephyr Open Source RTOS for IoT. <https://www.eeworldonline.com/zephyr-open-source-rtos-for-iot/>, September 2018.
- [61] Matthias Wenzl, Georg Merzdovnik, Johanna Ullrich, and Edgar Weippl. From hack to elaborate technique—a survey on binary rewriting. *ACM Comput. Surv.*, 52(3), June 2019.